

Bidirectional context-free grammar parsing for natural language processing[★]

Giorgio Satta^{a,*}, Oliviero Stock^b

^a *Institute for Research in Cognitive Science, University of Pennsylvania, 3401 Walnut Street,
Philadelphia, PA 19104-6228, USA*

^b *IRST, Istituto per la Ricerca Scientifica e Tecnologica, 38050 Povo, Trento, Italy*

Received March 1992; revised March 1993

Abstract

While natural language is usually analyzed from left to right, bidirectional parsing is very attractive for both theoretical and practical reasons. In this paper, we describe a formal framework for bidirectional tabular parsing of general context-free languages, and some applications to natural language processing are studied. The framework is general and permits a comparison between known approaches and the algorithms outlined here. A detailed analysis of the *redundancy problem* is given and a technique for improving the performance of bidirectional tabular parsers, whilst maintaining the flexibility of bidirectional strategies, is described. An algorithm for head-driven parsing and a general algorithm for island-driven parsing are studied. The former allows analyses of each constituent to be triggered by some fixed immediately dominated element, chosen on the basis of its information content. The latter permits analyses to start from any dynamically chosen positions within the input sentence, combining bottom-up and top-down processing without redundancy.

1. Introduction

A number of natural language parsing algorithms have been proposed since the field has been established. For a certain time it has especially stressed the

[★] The largest part of the present research was done while the first author was at Istituto per la Ricerca Scientifica e Tecnologica (IRST). This work has been developed as part of MAIA, the integrated AI project at IRST.

* Corresponding author, current address: Università di Venezia, Scienze dell'Informazione, via Torino 155, 30172 Mestre, Venezia, Italy e-mail: satta@moo.dsi.unive.it.

fact that natural languages are quite irreconcilable to more understood formal languages. This was certainly a consequence of the approach to language analysis derived from Transformational Theory [5] and bore some consequences on parsing itself. In more recent times there has been emphasis put on accepting the fact that natural language can be better described as *similar* to context-free languages, either with some restricted mechanism for taking care of the non context-free phenomena or with some special device included in the grammar specification so as to treat within the context-free apparatus all phenomena, at the cost of amplifying substantially the number of rules. Among many examples, see for instance the case of feature structure-based formalisms [36], based on augmentation of context-free rewriting; see also generalized phrase structure grammar [10] as an example of a linguistic theory not exceeding the generative power of context-free grammars. With context-free grammar based formalisms arising as the more sensible way to proceed, the natural language community began to appreciate the power of tabular parsing methods. It did so first developing its own approaches, the best known of which has been active chart parsing [18,19], then also confronting itself with the standard literature on context-free formal language parsing.

As in the case of formal languages, natural language has also been normally analyzed in a left-to-right fashion. Yet, a recent and pervasive trend in linguistic representation, by some meant as complementary to the context-free oriented thrust, by others as an independent basic concept for natural language, is the idea of information stored in the lexicon so as to control the characteristics of possible constituents that are to appear connected to the given word. This idea, not entirely new and already present for instance in categorial grammars [3] has led to the development of the concept of heads: symbols within the right-hand side of rewriting rules that bear the most relevant information in that rule, as for instance in the case of a verb within a verb phrase rule. Typically, the information attached to the lexical entry of the head constrains elements in the right-hand side of a rule. A parser should exploit this information to eliminate unnecessary searching. However, heads can appear in any position within the rules' right-hand side. In the head-driven parsing paradigm, the problem is solved by starting analysis from the head of a rule. The analysis then proceeds in a head-outward fashion, as opposed to traditional mono-directional parsing techniques. Some natural language processing systems based on modern syntactic theories have successfully adopted head-driven parsing strategies, either directly, as for instance in [6,27], or indirectly, by simulating it with a left-to-right strategy based on some kind of delayed evaluation, as for example in [41]. Comparisons with mono-directional parsing have shown how head-driven strategies succeed in restricting the search space by activating the maximum number of constraints.

If one adopts a linguistic neutral position, an information theoretic notion of head still turns out to be of major importance. In order to improve parsing performance, each head can be selected as the element within the right-hand side of a production that maximizes the conditional probability of having used

that production, given the presence of that element in the input string. This idea also fits in a stochastic context-free parsing paradigm. If probabilities associated with grammar productions are inferred from large corpora representing the domain of interest, head-driven parsing will minimize the average number of unnecessary partial analyses, which is normally due to occurrences of same categories in different right-hand sides. Of course, there is no *a priori* reason to expect that head elements selected in this way are always located in the leftmost position of each production right-hand side. Such a statistically oriented approach has been taken, for example, in [12]: preliminary tests of comparison with mono-directional strategies have again shown improved parsing performance.

Another important aspect that has been appreciated within the natural language community dealing with problems of speech recognition and speech understanding (but the same holds for handwritten recognition) is that the input string can be composed of words whose identity is not certain. The lower level of analysis may not be able to provide the parser with reasonable candidates or will give a very low degree of certainty. In these cases left-to-right strategies are a major limitation, because the parser can guess missing words only on the basis of their left context. Also, heavy search is required in case of missing words at the beginning of the utterance. So called island-driven parsers solve the problem by starting the analysis of the input sentence from several (dynamically determined) positions within it, called islands, that have been associated with the highest degree of likelihood, and then proceed outward from these positions. The island-driven parsing paradigm has long been applied in speech recognition/understanding systems, starting with early works in [45,47] to more recent ones, for instance [11,29]. Such a paradigm is considered of great relevance for applications in this field. Island-driven parsing strategies are also compatible with recent results for word-spotting techniques. Strong motivations for the use of island-driven strategies in speech recognition applications are presented in [48], based on the fact that island-driven flexibility allows the employment of optimal heuristics. These heuristics, when used with mono-directional strategies, do not guarantee admissibility. Island-driven parsing is also defended in [42], where the predictive power of bidirectional strategies is discussed, resulting from the ability to surround at both sides positions within the input having a high degree of uncertainty. Other advantages for speech processing applications are discussed in [29], where experiments of noisy speech and incomplete lattice parsing are reported, showing the increased flexibility of island-driven strategies as compared with mono-directional parsing.

In a similar vein, another interesting application for island-driven strategies is (written) ill-formed input. In fact, most successful techniques employed in error correcting/recovering parsing try to repair the input string on the basis of left analysis, and make crucial use of fixed length look-ahead in order to restrict the search space. Island-driven strategies generalize the use of look-ahead in an obvious way, providing error correction routines with partial analyses surrounding an incomplete (or missing) constituent. Therefore these

strategies can be effective in improving robustness of the parser in applications where non-standard input needs to be taken into account.

This paper discusses parsing techniques that extend the flexibility and the power of traditional tabular methods. The contributions of our work are the following:

- We propose an innovative approach and a systematization for bidirectional context-free tabular parsing. We show how different bidirectional strategies can be implemented using a basic computational scheme along with a class of transformations defined on the input grammar.
- Using such a framework, we abstract away some general computational properties of bidirectional parsing strategies. We show that under a worst case perspective the increased flexibility of bidirectional parsing requires more processing, as compared with mono-directional parsing. This fact has already been noted informally in the natural language parsing literature. We give a quantitative account of the problem.
- We then move from a worst case perspective to an average case perspective. We present an innovative technique that is effective in improving computational performance of bidirectional parsing. We propose both a head-driven tabular recognizer and an island-driven one, that have average case performance comparable with the one of well known mono-directional tabular parsers, but with the flexibility we require.

The paper is organized in the following way. In Section 2 we start by reviewing left-to-right tabular methods for the recognition of context-free languages. We consider these methods as based on “easy to process” grammars that cover the general form input grammar, following the approach proposed in [22]. The approach is then extended in Sections 3 and 4 to define some classes of “bidirectional” covering grammars, in order to study the parsing applications outlined above. In these two sections, the covering framework is used to discuss and compare different bidirectional methods found in the literature and to identify, through a worst case analysis, some inherent computational limitations of bidirectional strategies. Crucially, we observe how these limitations might affect the parsing process in the average case, overriding the advantages discussed above. We repair by developing and evaluating a general technique that overcomes the problem, and design a head-driven tabular recognizer and an island-driven one as an application. We conclude with some general comments.

The remaining part of this introduction is devoted to the notation and basic definitions that are used throughout this paper (the reader is also referred to [2,39]). Given a finite alphabet $\mathcal{A}$, we write $|\mathcal{A}|$ to represent the number of symbols in $\mathcal{A}$ and $\mathcal{P}(\mathcal{A})$ to represent the power set of $\mathcal{A}$. A *string* over $\mathcal{A}$ is a finite sequence of symbols from $\mathcal{A}$. Given a string w , we write $|w|$ to denote the length of w . The *null string* ε is the unique string whose length equals zero. The set of all strings over $\mathcal{A}$ is denoted $\mathcal{A}^*$ (ε included). For natural numbers i and j , let $w = a_{i+1}a_{i+2}\cdots a_j$ be a string in $\mathcal{A}^*$; if $j = i$, w represents ε .

A *context-free grammar* (CFG) is a four-tuple $G = (N, \Sigma, P, S)$, where N is a

finite set of *nonterminal* symbols, Σ is a finite set of *terminal* symbols such that $N \cap \Sigma = \emptyset$, P is a finite set of *productions* and $S \in N$ is a special symbol called *start symbol*. Productions in P are denoted by symbols p_r , $1 \leq r \leq |P|$, and have the form $D_r \rightarrow Z_{r,1}Z_{r,2} \cdots Z_{r,\pi_r}$, $\pi_r \geq 0$, where $D_r \in N$ and $Z_{r,j} \in N \cup \Sigma$, $1 \leq j \leq \pi_r$. A production $p_r \in P$ is called a *null* production if $\pi_r = 0$; it is called a *chain* production if $\pi_r = 1$, $Z_{r,1} \in N$. We write π_G to denote the maximum among all π_r , $1 \leq r \leq |P|$.

A binary relation *derive*, written $\Rightarrow_G$, is associated with G and is defined in the following way: for $\gamma, \delta \in (N \cup \Sigma)^*$, $\gamma A \delta \Rightarrow_G \gamma \alpha \delta$ whenever $A \rightarrow \alpha$ belongs to P . We will also use the reflexive and transitive closure of the derive relation, written $\stackrel{*}{\Rightarrow}_G$. The language $L(G)$ derived by G is the set $\{w \mid S \stackrel{*}{\Rightarrow}_G w, w \in \Sigma^*\}$. G is *reduced* if for every $X \in N \cup \Sigma$ we have $S \stackrel{*}{\Rightarrow}_G uXv \stackrel{*}{\Rightarrow}_G uvw$ for some $u, v, w \in \Sigma^*$, that is there are no useless symbols in G .

Throughout this paper we will be concerned with the problem of how difficult it is to process grammar-string pairs $\langle G, w \rangle$: we will always assume the random access machine model of computation and our analyses will be carried out under the *uniform cost criterion* (see [1]). The size of a CFG G is defined as follows:

$$|G| = \sum_{A \rightarrow \alpha \text{ in } P} |A\alpha|.$$

We will also use the following standard notation. Given two functions f and g defined on natural numbers, we write $f = O(g)$ whenever $f(n) \leq cg(n)$ for some constant $c > 0$ and for almost all n ; we write $f = \Theta(g)$ whenever $c_1g(n) \leq f(n) \leq c_2g(n)$ for some constants $c_1, c_2 > 0$ and for almost all n .

As a final note, in this paper we will consider the *recognition (membership)* problem for context-free languages, that is the problem of deciding whether an input string belongs to the language generated by an input grammar. Within the perspective of the tabular methods explored here, this problem is very closely related to the parsing problem. In fact, the content of a recognition table can be viewed as a representation of the parsing forest associated with the input string, and standard algorithms exist in the literature that retrieve parse trees from such a table (see for instance [2]).

2. Left-to-right tabular methods

Tabular methods represent an important family among algorithms for recognition/parsing of general context-free grammars. Most popular in this family are the Cocke–Kasami–Younger (CKY) algorithm [50] and the Earley algorithm [9]. An important improvement of the Earley algorithm was presented by Graham, Harrison and Ruzzo [14]. This will be discussed below. An interesting subfamily of tabular methods called *C-parsers*, recently discussed in [22], is also linked to the Earley algorithm. All these tabular methods exhibit some obvious similarities which will be briefly discussed (see also [13]).

2.1. Recognition invariants

Given a reduced context-free grammar $G = (N, \Sigma, P, S)$ and a sentence $w = a_1 \cdots a_n$, $w \in \Sigma^*$ and $n \geq 1$, tabular methods construct a recognition matrix T , which is a $(n+1) \times (n+1)$ square matrix. In the general case, entries of T are filled in such a way that the following condition (invariant) holds (0-origin addressing is used for T):

$$A \in t_{i,j} \text{ iff } S \xRightarrow{*} \alpha A \beta \xRightarrow{*} \alpha a_{i+1} \cdots a_j \beta, R_T(\alpha, \beta, w, i, j), \quad (1)$$

where $0 \leq i \leq j \leq n$, $A \in N$, $\alpha, \beta \in (N \cup \Sigma)^*$. Relation R_T in (1) varies with different methods and it functions as a filter on elements in the entries of T . In fact R_T represents the prediction and/or look-ahead techniques employed by the parser ($R_T \equiv \text{true}$ in the case of the CKY algorithm). Instances of (1) will be discussed for the algorithms presented in Sections 3 and 4.

2.2. Parsing efficiency

In natural language applications it is often the case that $|G|$ is much bigger than the length n of the input sentence. We discuss the efficiency of tabular parsing methods both from the point of view of grammar size and of sentence length.

The CKY algorithm is perhaps the simplest tabular parsing method. It requires the input grammar G to be cast in *Chomsky normal form* (CNF; see [2]). The time bound for the CKY algorithm is $O(|G_C|n^3)$, where G_C is the converted grammar. Converting G in CNF may result in quadratic size expansion [39]. Hence, with respect to the input grammar, running time for the CKY algorithm is $O(|G|^2n^3)$.

The well-known Earley algorithm has worst case running time of $O(|G|^2n^3)$ (see [8]), where G is a general form context-free grammar. An improvement of the Earley algorithm has been proposed in [14], resulting in a method with time bound $O(|G|n^3)$. This method makes use of a binary operation that Graham et al. have called *star-product*. Along with some interesting grammar representations discussed in the cited work, the implementation of the *star-product* operation is central to the run time improvement. Throughout this paper, this result will be taken as the term of comparison in discussing time efficiency for the bidirectional parsing applications that will be presented.¹

2.3. Covering grammars

Let G and G_c be two CFGs such that $L(G) = L(G_c)$. Grammar G_c is said to be a *cover* for G (to *cover* G) if each parse tree of w in G can be “recovered” from some parse tree of w in G_c . In the standard literature, the notion of

¹ The $O(|G|n^3)$ upper bound is not the best known: Valiant [43] presents an asymptotical improvement. This method, however, is impractical and is never used in natural language applications, because of the large constants hidden in the asymptotical notation.

recovery is specified by means of a homomorphism from productions in G_c to productions in G (see [23]). For the purpose of this paper, we extend this notion by requiring linear processing. A context-free grammar G is said to be in *bilinear form* if every production of G contains no more than two nonterminal symbols within its right-hand side.

As recently pointed out by Leermakers [22], many parsing algorithms for general CFGs work with a covering grammar G_c computed on the fly from the input grammar G . It turns out that G_c is in a particular kind of bilinear form. More precisely, G_c is in bilinear form and for every production $A \rightarrow \alpha$ in G_c we have $|\alpha| \leq 2$ (G_c admits null-productions. See also [34] for discussion). This important observation gives more insight into the relation between input grammar and parsing efficiency. It will be used in the following sections to derive major limitations for bidirectional parsing with tabular methods.

As an example, and for further comparisons, we will discuss here the class of grammar covers used by the Earley algorithm. The intuitive idea underlying the following definition is that the right-hand side of a context-free production $p_r: D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ is divided into two parts $Z_{r,1} \cdots Z_{r,s}$ and $Z_{r,s+1} \cdots Z_{r,\pi_r}$, for every $0 \leq s \leq \pi_r$ (in the standard literature this is also represented by means of symbols called *dotted rules*, having the form $\langle D_r \rightarrow Z_{r,1} \cdots Z_{r,s} \cdot Z_{r,s+1} \cdots Z_{r,\pi_r} \rangle$).

Definition 1. Let $G = (N, \Sigma, P, S)$ be a CFG. For every production p_r in P and for every s , $0 \leq s \leq \pi_r$, $I_r^{(s)}$ is an *e-item*.

In the following, symbol $I_G^{(E)}$ will represent the set of all e-items associated with G .

Given an input grammar G , the Earley algorithm processes strings in Σ^* according to a cover G_E built on the fly from G . The following definition states the relation between G and G_E . (Without loss of generality, we assume that $p_1 \in P$ is the only production that rewrites S .)

Definition 2. Let $G = (N, \Sigma, P, S)$ be a CFG. The *Earley cover* (*e-cover* for short) $G_E = (I_G^{(E)}, \Sigma, P_E, I_1^{(\pi_1)})$ is defined as follows:

- (i) for every production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ in P , $\pi_r \geq 1$, and for every e-item $I_r^{(s+1)}$, $0 \leq s < \pi_r$:
 - (a) if $Z_{r,s+1} \in N$, the production

$$I_r^{(s+1)} \rightarrow I_r^{(s)} I_u^{(\pi_u)}$$
 belongs to P_E for every $p_u \in P$ such that $D_u = Z_{r,s+1}$;
 - (b) if $Z_{r,s+1} \in \Sigma$, the production

$$I_r^{(s+1)} \rightarrow I_r^{(s)} Z_{r,s+1}$$
 belongs to P_E ;
- (ii) for every e-item $I_r^{(0)}$, the production

$$I_r^{(0)} \rightarrow \varepsilon$$
 belongs to P_E .

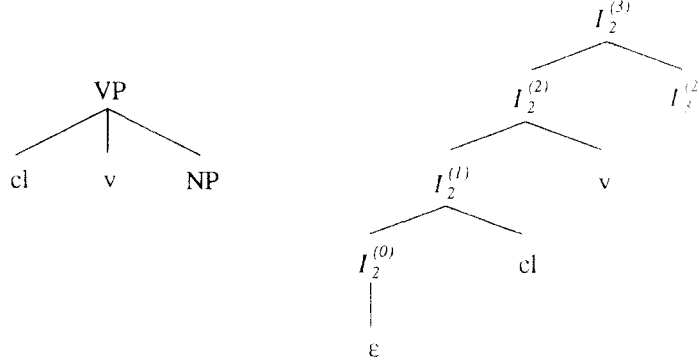


Fig. 1. Schematic representation of production $VP \rightarrow cl\ v\ NP$ and its corresponding productions in the associated Earley cover.

Each nonterminal of G_E can be thought of as representing a partial analysis of a nonterminal of G , according to some production in P . It is not difficult to verify that G_E is in fact a cover for G .

Example 3. We introduce a small grammar that captures the word order exhibited by romance languages in sentences with pronominal clitics. An English sentence such as *John gives a book to her* is usually expressed by the word order *John to-her gives a book*, where *to-her* is a pronominal clitic. Let $G_{cl} = (N, \Sigma, P, S)$, $N = \{S, VP, NP\}$, and $\Sigma = \{cl, det, n, v\}$. We use the following productions:

- (1) $S \rightarrow NP\ VP$,
- (2) $VP \rightarrow cl\ v\ NP$,
- (3) $NP \rightarrow det\ n$.

In the corresponding Earley cover, production 2 is “covered” by the following productions (see Fig. 1):

$$\begin{aligned} I_2^{(0)} &\rightarrow \varepsilon, & I_2^{(2)} &\rightarrow I_2^{(1)}\ v, \\ I_2^{(1)} &\rightarrow I_2^{(0)}\ cl, & I_2^{(3)} &\rightarrow I_2^{(2)}\ I_3^{(2)}. \end{aligned}$$

The Earley algorithm fills in the entries of the recognition matrix T according to the condition expressed in (1) recast for G_E , where $R_T(\alpha, \beta, w, i, j) \equiv (\alpha = a_1 \cdots a_i)$. This property is called the valid prefix property in [33]. It imposes the compatibility of the analyzed constituent with the already processed prefix of the input string. In bidirectional parsing this property can be replaced by a similar “infix” condition. The recognition matrix T is processed in an iterative way for every entry $t_{i,j}$ and for every production $I_r^{(s+1)} \rightarrow I_r^{(s)} I_u^{(\pi_u)}$ in P_E , by means of the following condition:

$$I_r^{(s+1)} \in t_{i,j} \text{ iff } \bigvee_{k=i}^j (I_r^{(s)} \in t_{i,k} \wedge I_u^{(\pi_u)} \in t_{k,j}). \quad (2)$$

Such a condition is an abstract description of the fundamental step in the Earley algorithm, called the *completer* step. Two other steps are found in the Earley algorithm, called *scanner* and *predictor*, that deal with productions of the form (i)b and (ii) respectively in Definition 2. These steps do not affect the worst-case time complexity of the algorithm.

From Definitions 1 and 2 one can derive $|I_G^{(E)}| = |G|$ and $|P_E| = O(|G||P|)$; in the worst case, we have $|P_E| = \Theta(|G|^2)$ (it is not difficult to exhibit such a worst-case grammar). Given the search process implied by Condition (2), this fact reveals one of the major limitations of the original version of the Earley algorithm. For every $A \in N$, let $\langle I_A \rangle = \{I_r^{(\pi_r)} \mid p_r : A \rightarrow Z_{r,1} \cdots Z_{r,\pi_r} \in P\}$. Items from $\langle I_A \rangle$ that are simultaneously present in an entry $t_{i,j}$ of the recognition matrix are all processed by the completer step independently one of the other, notwithstanding the fact that the results of these processes are identical. This suggests we can improve the Earley algorithm simply by representing every item in $\langle I_A \rangle$ by means of a unique symbol I_A , resulting in an optimization of the cover G_E such that $P_E = O(|G|)$. In this way, Condition (2) can be executed in $O(|G|(j-i))$ elementary steps, and the running time of the improved algorithm is $O(|G|n^3)$. It is worth noting here that the already mentioned star-product operation defined in [14] computes on the fly sets $\langle I_A \rangle$ for every $A \in N$ and for each entry $t_{i,j}$. Such a computation is the basis for the $O(|G|n^3)$ upper bound obtained by the cited authors, as we have shown with the covering argument above.

As a final observation, the proposed introduction of symbols I_A , $A \in N$, has no consequence for standard parsing algorithms in which parse trees of the input sentence are constructed under the hypothesis that nonterminals of the cover are stored in T along with a list of pointers to those entries which caused the new entry to appear in the recognition matrix. We will therefore use similar equivalence classes in the covering grammars that will be introduced in the next sections.

2.4. Mono-directional versus bidirectional parsing

The right-hand term in Condition (2) represents the fundamental step for most tabular parsing methods, given the fact that these methods use some kind of bilinear cover for the input grammar. For this condition, it makes no sense to say that a binary production of the form $A \rightarrow BC$ entails a left-to-right (or a right-to-left) way of parsing *per se*. In fact there exist tabular parsing methods that construct the nonterminal A either starting from B and then proceeding to the right, or starting from C and then proceeding to the left, depending on which of the two symbols B and C has been constructed after the other (see for example the chart parsing methods described in [19]). For the same reason, the standard version of the CKY algorithm ([2,15]) is not, *strictu sensu*, a mono-directional parsing method. For the Earley algorithm, nevertheless, the cover G_E forces a left-to-right order for the analysis of each nonterminal in N ,

resulting in the overall left-to-right parsing strategy with respect to the input grammar G . This extends to all parsing methods that use covers of this kind.

The term “bidirectional parser” refers to all those methods that are able to analyze strings of grammar symbols by proceeding in both directions. For tabular parsing methods, this translates into the ability to extend partial analyses of grammar productions at both sides. The adoption of bilinear covers qualitatively different from the ones discussed in this section is proposed as the right move in order to weaken the intrinsic mono-directionality of most tabular parsing methods, resulting in bidirectional performance. Changing the cover affects the efficiency of the parsing process, as we will discuss in the next sections.

3. Head-driven parsing

As already mentioned in the introduction, in natural language processing the notion of head has been inherited from modern theories of grammar. According to these theories, the head individuates the categorial labels of nodes that dominate it, up to two or three levels above, and provides a general skeleton for the related branches. The head also carries categorial and thematic information about other elements within the constituent (see for example [10,16,26]). From a computational perspective, it turns out that on the basis of their functional information content, heads constrain the acceptability of elements to their left as well to their right.

The head-driven paradigm, also called head-outward, head-corner or head-first, has been investigated by many researchers working on natural language processing, both from the point of view of parsing (see among others [20,27,31,38]) and of generation ([37,44]). In the following, a specific framework based on the idea of covering grammars will be introduced and used to establish a comparison between the different proposals found in the literature and to discuss major limitations of head-driven parsing with respect to mono-directional parsing. Such a framework will also be employed in the specification of a head-driven tabular recognizer. A selective technique is used to prevent, in the average case, the shortcomings of bidirectional parsing, but maintain the flexibility of this paradigm.

3.1. Head covers

Let G be a context-free grammar. Assume that every non-null production in G contains exactly one selected element within its right-hand side, to be called *head*. We define *head-driven* any recognition/parsing strategy in which analyses are worked out starting from head elements within the sentence and then proceeding outward in some way. A head-driven parser for CFGs can be produced by assuming that the analysis of each production found within a derivation of the input string is triggered by the completion of the analysis of the *fixed* head element of its right-hand side. Due to the fact that the

head element is not necessarily the leftmost or the rightmost one within the right-hand side, partial analyses need to be extended at both sides, i.e., the parser should employ some bidirectional technique. In the case of a tabular method, both the head condition and the bidirectional requirement can be accomplished by defining an appropriate cover for the input grammar.

For simplicity, we assume that G has no null production. Appendix A shows how to drop this restriction. For every production p_r of G having the form $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$, $\pi_r \geq 1$, we define τ_r , $1 \leq \tau_r \leq \pi_r$, to be the index of the designated head element within the right-hand side of p_r . The following definition introduces the nonterminal symbols of the studied cover.

Definition 4. Let $G = (N, \Sigma, P, S)$ be a context-free grammar. The *h-items* associated with G are all and only the elements defined as follows:

- (i) for every production $p_r \in P$, $I_r^{(s,t)}$ is an h-item, $0 \leq s < \tau_r \leq t \leq \pi_r$, $(s, t) \neq (0, \pi_r)$;
- (ii) for every $A \in N$, I_A is an h-item.

In the following, symbol $I_G^{(H)}$ will denote the set of all h-items associated with G . In the next definition the idea is to cover G in the most general way, given the set of nonterminals $I_G^{(H)}$. Without loss of generality, we assume that only one production in G rewrites the start symbol S .

Definition 5. Let $G = (N, \Sigma, P, S)$ be a context-free grammar. The *head cover* (*h-cover* for short) $G_H = (I_G^{(H)}, \Sigma, P_H, I_S)$ for G is a context-free grammar whose production set P_H is partitioned in the following way:

- (i) the *projection set* $P_H^{(1)}$ is defined as follows:
 - (a) for every production $D_r \rightarrow Z_{r,1}$ in P , the production $I_{D_r} \rightarrow X_H$ belongs to $P_H^{(1)}$, where $X_H = I_A$ if $Z_{r,1} = A \in N$ and $X_H = Z_{r,1}$ if $Z_{r,1} \in \Sigma$;
 - (b) for every production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ in P such that $\pi_r > 1$, the production $I_r^{(\tau_r-1, \tau_r)} \rightarrow X_H$ belongs to $P_H^{(1)}$, where $X_H = I_A$ if $Z_{r,\tau_r} = A \in N$ and $X_H = Z_{r,\tau_r}$ if $Z_{r,\tau_r} \in \Sigma$;
- (ii) the *expansion set* $P_H^{(2)}$ is defined as follows. For every production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ in P such that $\pi_r > 1$:
 - (a) for every h-item $I_r^{(s,t)}$, the productions (when defined) $I_r^{(s,t)} \rightarrow X_H I_r^{(s+1,t)}$, $I_r^{(s,t)} \rightarrow I_r^{(s,t-1)} Y_H$ belong to $P_H^{(2)}$, where $X_H = I_A$ if $Z_{r,s+1} = A \in N$ and $X_H = Z_{r,s+1}$ if $Z_{r,s+1} \in \Sigma$ (a similar condition holds for symbol Y_H with respect to $Z_{r,t}$);

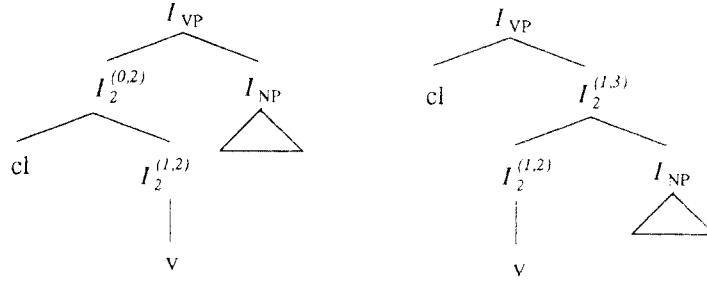


Fig. 2. Production $VP \rightarrow cl \vee NP$ of G_{cl} with head element v gives rise to a structural ambiguity in the analysis of symbol I_{VP} in the associated h-cover.

(b) the productions

$$\begin{aligned} I_{D_r} &\rightarrow X_H I_r^{(1, \pi_r)}, \\ I_{D_r} &\rightarrow I_r^{(0, \pi_r - 1)} Y_H \end{aligned}$$

belong to $P_H^{(2)}$, where the same condition as in (ii)a holds for symbols $X_H, Y_H \in I_G^{(H)} \cup \Sigma$ with respect to symbols $Z_{r,1}, Z_{r,\pi_r}$.

Example 6. Grammar G_{cl} has been introduced in Example 1; here we exhibit one among the possible head covers for such a grammar. Let us assume for G_{cl} , $\tau_1 = 2$, $\tau_2 = 2$, $\tau_3 = 1$, that is VP , v and det are the heads of productions 1, 2 and 3 respectively. Then the h-cover associated with G_{cl} contains the following productions:

$$\begin{aligned} P_H^{(1)}: \quad & I_1^{(1,2)} \rightarrow I_{VP}; & I_2^{(1,2)} &\rightarrow v; & I_3^{(0,1)} &\rightarrow det; \\ P_H^{(2)}: \quad & I_S \rightarrow I_{NP} I_1^{(1,2)}; & I_2^{(0,2)} &\rightarrow cl I_2^{(1,2)}, & I_{NP} &\rightarrow I_3^{(0,1)} n. \\ & & I_2^{(1,3)} &\rightarrow I_2^{(1,2)} I_{NP}, \\ & & I_{VP} &\rightarrow I_2^{(0,2)} I_{NP}, \\ & & I_{VP} &\rightarrow cl I_2^{(1,3)}; \end{aligned}$$

Observe that there are different ways of expanding the head v in the analysis of production $VP \rightarrow cl \vee NP$ of G_{cl} . This results in the ambiguity of the associated h-cover and in the undesired “redundancy” in the analysis of constituent VP (see Fig. 2); the problem will be discussed at length in the next section.

The cover G_H is in the particular bilinear form discussed in Section 2.3. As it can be verified from the definition, a tabular parsing method that weakens the usual left-to-right control and that works on G_H will parse $L(G)$ in the desired way. The bidirectional behaviour of such a parser is the most general one, in the sense that any order is permitted in the expansion of a head. In the next section some consequences of such a general approach will be drawn. Different head-driven methods that have already been proposed in the literature will also be discussed as particular cases of the one presented here.

3.2. Parsing efficiency

In this section we address the issue of parsing efficiency for head-driven strategies. From the definition of h-item we can derive $|I_G^{(H)}| = |N| + \sum_{k=1}^{|P|} [\tau_k(\pi_k - \tau_k + 1) - 1]$; this implies that the number of elements in $I_G^{(H)}$ is $\Theta(\pi_G|G|)$ (recall that π_G is the maximum length of the right-hand side of all productions in G). Note that $\Theta(\pi_G|G|)$ is also a bound for P_H . In fact, in the general case there are two different ways of rewriting a symbol of the form $I_r^{(s,t)}$ in G_H , and overall, all symbols of the form I_A give rise to no more than $2|P|$ productions. Some consequences of this fact will be derived below.

As a first observation, $|I_G^{(H)}| = \Theta(\pi_G|G|)$ means that in order to recognize a string of length n , even the most efficient of the tabular methods discussed in Section 2 will take, in the worst case, an amount of time at least proportional to $\pi_G|G|n^3$. In fact, in cases of maximal ambiguity such an algorithm will have to insert $\Theta(\pi_G|G|)$ different symbols of the form $I_r^{(s,t)}$ in each entry $t_{i,j}$, $i < j$, of the recognition matrix. Moreover, the check associated with each insertion will take $\Theta(j - i)$ steps in the worst case.

It is possible to improve such a performance by substantially reducing $|I_G^{(H)}|$ without altering the recognized language. This can be done by means of choosing a preferred order in the expansion of each head element in G . In [20,40], chart parsing methods are presented that are examples of such a solution. The authors propose to expand every head in G in one direction, say rightward, until the rightmost symbol in the corresponding production is reached, and then proceed with the left expansion of the partial analysis so obtained. This solution is based on the assumption that a privileged order of expansion can be selected independently of the input string, i.e., such a choice is established once for all by the grammar writer. There are applications in which this assumption may be inappropriate. In stochastic context-free grammar parsing, for example, the most convenient strategy is to extend a partial analysis at one side, only if the extension will take the score of the obtained partial analysis above a specified threshold, and this depends upon the recognized input string.

A second consequence of the definition of the h-cover G_H concerns the parsing time performance in the average case. From Definition 2 we see that in the Earley cover G_E every production $p_r \in P$ is “covered” by a unique leftmost derivation of symbol $I_r^{(\pi_r)}$, having length $\pi_r + 1$; in other terms, the right-hand side of p_r is an unambiguous string with respect to G_E . On the contrary, the right-hand side of p_r is an ambiguous string for G_H . All leftmost derivations in G_H of the right-hand side of p_r can be seen as paths from symbol I_D , to symbol $I_r^{(\tau_r-1, \tau_r)}$ in the diagram of Fig. 3. It turns out that, if a tabular technique is used, it takes $O(\pi_r^2)$ steps to completely analyze symbol D_r by means of production p_r , working with G_H , while the same analysis takes $O(\pi_r)$ steps in the case of cover G_E . This is exemplified in Fig. 4 where the left-to-

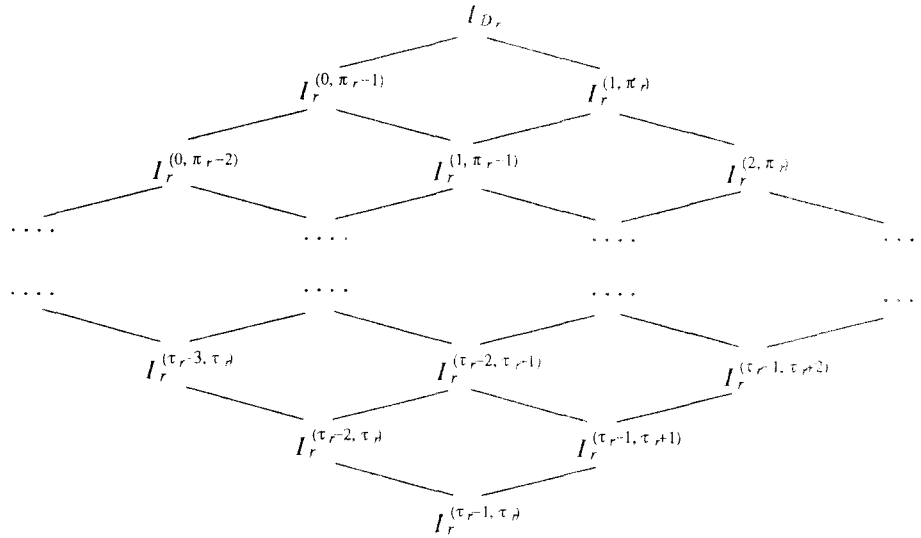


Fig. 3. Each path from symbol I_{D_r} to symbol $I_r^{(\tau_r-1, \tau_r)}$ in the diagram corresponds to a possible leftmost derivation in G_H of the right-hand side of a production $D_r \rightarrow Z_{r,1} \dots Z_{r,\pi_r}$ in G having head at Z_{r,τ_r} .

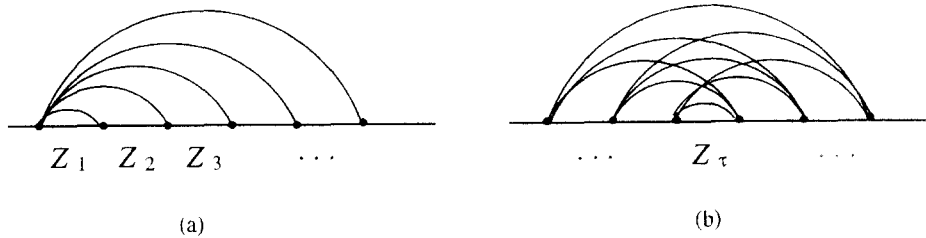


Fig. 4. Comparison between left-to-right (a) and head-driven (b) recognition of a production right-hand side by means of a tabular method. In each graph an arc represents a partial analysis (a nonterminal of the used cover) in some entry of the recognition matrix.

right and the head-driven bidirectional cases are compared (the content of the upper triangular part of the recognition matrix is represented by means of an undirected graph). As can be seen from Fig. 4(b), the additional computational effort in the head-driven case is due to the redundancy of the cover G_H . Every pair of h -items $I_r^{(s,t)}$ and $I_r^{(s',t')}$ such that $s < s' < t < t'$ represents a partial duplication in the analysis if both items share the same analyzed constituents $Z_{r,s'+1}, \dots, Z_{r,t}$. This worsens the performance of a tabular method even in the case of an unambiguous input grammar.

This reveals a problem that has already been noticed in the natural language parsing literature (see, among others, [31,48]): the main shortcoming of head-driven parsing is (partial) analysis redundancy. In the next section, a bidirectional tabular method is proposed that overrides the redundancy of the

cover G_H , still maintaining the general head-driven parsing strategy previously discussed, i.e., the capability of deciding at run-time at which side to extend each partial analysis.

3.3. A head-driven tabular recognizer

As an application of the head cover definition $G_H = (I_G^{(H)}, \Sigma, P_H, I_S)$ discussed in the previous sections, we present here a head-driven tabular recognizer whose specification entirely rests upon G_H . Such an algorithm recognizes the set $L(G)$, where G is the context-free grammar in null production free form being covered by G_H . This method witnesses the $O(\pi_G |G| n^3)$ time upper bound claimed in the previous section. The recognizer works on G_H in a pure bottom-up fashion. Partial analyses are obtained starting from every head element that is found within the input string, as specified by the cover. Recognition then proceeds by outward expansion.

Two major steps are found within the studied algorithm, that are called *left-expand* and *right-expand*. In the general case, these steps insert a symbol $A_I \in I_G^{(H)}$ into element $t_{i,j}$, $i < j$, of a recognition matrix T whenever, for some production $A_I \rightarrow X_H Y_H$ in $P_H^{(2)}$, the disjunction $\bigvee_{k=i+1}^{j-1} (X_H \in t_{i,k} \wedge Y_H \in t_{k,j})$ holds. The two steps are triggered by the unique element between X_H and Y_H that derives the head of the corresponding production in G . Productions in $P_H^{(1)}$ are instead used by the *init* step and the *project* step, which trigger partial analyses from head elements in Σ or in N respectively. Apart from the *init* step, the execution of the remaining steps above on an h-item in T can take place in any order and no restriction is made on the order in which h-items in T are processed. The issue of control has been left out of the algorithm, in order to emphasize its flexibility. As usual, an agenda data structure can be employed to implement a chosen policy.

The analysis proliferation problem due to the bidirectionality of the cover G_H is dealt with by extra bookkeeping as follows. Every time we decide to “extend” an h-item $I_r^{(s,t)} \in t_{i,j}$ at one side, by means of the application of the left-/right-expand steps, we record it by updating a set-valued symbol $Q(i, j, d)$, $d \in \{\text{left}, \text{right}\}$. In this way we are able to block further extensions of h-item $I_r^{(s,t)}$ at the opposite side. This technique is called *subsumption blocking* and was originally presented in [31]. It is easy to see that, in parsing G_H with the subsumption blocking technique, we are still allowed to choose any path in the diagram of Fig. 3, i.e., we can explore the input string in any direction while completing the analysis of the right-hand side of a production of the original input grammar. Once an extension order has been dynamically chosen, alternative orders that would have led to the formation of redundant analyses are blocked. This is exemplified in Fig. 5, where the general bidirectional case is compared with the result of applying the subsumption blocking technique. Although the worst case complexity previously discussed for G_H is not affected by the subsumption blocking technique, we have a considerable reduction

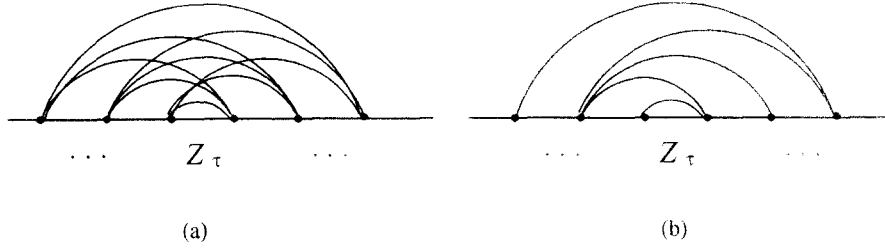


Fig. 5. Comparison between general bidirectional head-driven parsing (a) and head-driven parsing using the subsumption blocking technique (b), in the case of recognition of a production right-hand side. In case (b) each partial analysis has been extended at some randomly chosen side and further extension of that analysis at the opposite side has been blocked.

of partial analysis redundancy which improves parsing performance in the average case. Subsumption blocking acts in such a way that only one possible leftmost derivation can now be associated in G_H with the right-hand side of a production of the original input grammar. As will be discussed in Appendix B, this does not affect the completeness of the algorithm.

In the specification of the algorithm below, we use a predicate *partial*, defined on $I_G^{(H)}$, such that *partial*(A_H) holds if and only if h-item A_H has the form $I_r^{(s,t)}$. As a convention, each set-valued symbol is initially assigned the empty set. The algorithm is reported in the following (some code optimizations have been omitted, in order to render the algorithm more readable).

Algorithm 7.

Input. An h-cover $G_H = (I_G^{(H)}, \Sigma, P_H, I_S)$ for a context-free grammar G (with no null production); a string $w = a_1 \cdots a_n$ in Σ^* , $n > 0$.

Output. Accept/reject.

Method. An $(n+1) \times (n+1)$ matrix T is used, whose entries take values in $\mathcal{P}(I_G^{(H)})$; symbols $Q(i, j, d)$, $0 \leq i < j \leq n$, $d \in \{\text{left}, \text{right}\}$, are also used: they are assigned values in $\mathcal{P}(I_G^{(H)})$.

for every $i \in \{1..n\}$ do

comment: init step

for every production $A_H \rightarrow a_i$ do

$t_{i-1,i} := t_{i-1,i} \cup \{A_H\}$

endfor

endfor

for every A_H added to $t_{i,j}$, $0 \leq i < j \leq n$, do

comment: project step

for every production $B_H \rightarrow A_H$ do

$t_{i,j} := t_{i,j} \cup \{B_H\}$

endfor

comment: left-expand step

for every production $B_H \rightarrow X_H A_H$ do


```

if  $A_H \notin Q(i, j, left)$  then
  for every  $i' < i$  do
    if  $(X_H \in t_{i',i}, X_H \notin Q(i', i, right))$  or  $(X_H = a_i, i' = i - 1)$  then
       $t_{i',j} := t_{i',j} \cup \{B_H\}$ 
      if  $partial(X_H)$  then
         $Q(i', i, left) := Q(i', i, left) \cup \{X_H\}$  endif
      if  $partial(A_H)$  then
         $Q(i, j, right) := Q(i, j, right) \cup \{A_H\}$  endif
      endif
    endif
  endfor
endif
endif
endif
comment: right-expand step
for every production  $B_H \rightarrow A_H X_H$  do
  if  $A_H \notin Q(i, j, right)$  then
    for every  $j' > j$  do
      if  $(X_H \in t_{j,j'}, X_H \notin Q(j, j', left))$  or  $(X_H = a_{j+1}, j' = j + 1)$  then
         $t_{i,j'} := t_{i,j'} \cup \{B_H\}$ 
        if  $partial(X_H)$  then
           $Q(j, j', right) := Q(j, j', right) \cup \{X_H\}$  endif
        if  $partial(A_H)$  then
           $Q(i, j, left) := Q(i, j, left) \cup \{A_H\}$  endif
        endif
      endif
    endfor
  endif
endfor
endif
endif
if  $I_S \in t_{0,n}$  then accept else reject endif
end.

```

An invariant condition satisfied by the above recognizer is reported and discussed in Appendix B, from which the correctness of the method can easily be obtained. We study the running time of Algorithm 7. General computational complexity issues for head-driven bidirectional parsing have already been addressed in Section 3.2. The following result shows that the claimed $O(\pi_G |G| |w|^3)$ time upper bound is effective for cover G_H .

Theorem 8. *Let $f(G_H, w)$ be the running time of Algorithm 7. Then $f(G_H, w) = O(\pi_G |G| |w|^3)$.*

Proof. We consider membership, insertion and test *partial* to be elementary operations that can be carried out in constant time. Observe that $|P_H^{(1)}| = O(|G|)$. For given i and j , the init step and the project step never visit more than once each production in $P_H^{(1)}$. Then, the amount of time required by all possible executions of these steps is bounded by $O(|G| |w|^2)$.

The most complex steps turn out to be the left-expand and the right-expand. We recall that $|P_H^{(2)}| = O(\pi_G|G|)$. In the case of the left-expand step, for given i' , i and j , we have that each production in $P_H^{(2)}$ is visited at most once. Then the overall amount of time charged to the step is bounded by $O(\pi_G|G||w|^3)$. The same reasoning also applies to the right-expand step, and this proves the theorem. $\square$

We conclude the present section with some remarks. The upper bound reported above is tight, that is a sequence of grammar sentence pairs can be constructed that attains the worst case. In natural language processing applications, the context-free grammars that are commonly employed have maximum right-hand side length not greater than three or four. In such cases, the worst-case running time of the above method is quite close to the one of already known mono-directional recognizers. As far as average case performance is concerned, the problem of analysis redundancy is a crucial one. As already discussed in Section 3.2, this problem affects recognition in such a way that the worst-case dependency on the size of G reported by Theorem 8 is also found in the average case. Thus, the advantages of the adoption of a head-driven strategy are completely overridden. The subsumption blocking prescription we have proposed is effective for eliminating analysis redundancy. Starting from any immediate realization $Z_{r,1}, Z_{r,2}, \dots, Z_{r,\pi_r}$ of a constituent D_r within the input string, Algorithm 7 constructs D_r in no more than π_r steps. In practical cases, the resulting performance is not inferior than that of mono-directional recognizers, but, because of its flexibility, the algorithm can take advantage of situations in which information is not uniformly distributed among the symbols of the input sentence.

The above method works in a pure bottom-up fashion, as shown by the invariant presented in Appendix B. The adoption of top-down prediction in head-driven recognition has been proposed in [20] and is studied at length in [38] as a generalization of the left-corner parsing strategy (see for instance [28]). In the resulting recognizer, the head element of a constituent A is directly predicted by the head of a constituent B whenever A is an own argument of B itself. This move is responsible of an additional $|N|$ factor in the worst-case running time. On average, however, top-down prediction should improve the performance of the recognizer by further restricting the search space. As discussed in [38], in cases of natural language context-free grammars enriched with feature structure this strategy allows the earliest selection in the recognition of the arguments of the head having the desired features.

4. Island-driven parsing

Island-driven parsing strategies begin the analysis of the input sentence from several dynamically determined positions, and then proceed outward in both

directions. As already mentioned in the introduction, the notion of island-driven parsing, often called middle-out parsing, seems to originate from the early speech community. In the present section a specific definition for island-driven context-free grammar parsing is provided. The problem is viewed as a particular application of the general bidirectional parsing paradigm. Along the lines of Section 3, we introduce a specific framework based on the notion of covering grammar, and find that in the general case island-driven parsing may require substantially more computational effort than head-driven parsing. Different proposals for middle-out parsing from the literature are also discussed. Finally, an island-driven tabular recognizer is studied in detail. This method combines without any restriction bottom-up and top-down strategies and adopts a selective searching technique such that the construction of redundant partial analyses is entirely avoided.

4.1. Island covers

Let w be a string in Σ^* . For some integer M , $1 \leq M \leq |w|$, assume that M elements in different positions within w are selected, to be called *islands*. We define *island-driven* any recognition/parsing strategy that is forced to start analysis for w from the selected islands and then proceeds outward (this is adapted from the *middle-out* strategy defined in [49]).

In island-driven context-free parsing, the analysis of a constituent A within the input sentence can be triggered by the completion of one or more constituents (dominating the selected islands) in the right-hand side of a production for A . This means that the parser must be prepared to start partial analyses for a production p from any set of symbols in the right-hand side of p itself. This forces the parser to extend partial analyses for p at both sides, i.e., to analyze the input in a bidirectional fashion. In addition, partial analysis for p can now be extended by combination with other partial analyses for the same production. As opposed to head-driven parsing, in island-driven parsing one must also deal with cases in which no island at all has been selected within the portion of the input where a constituent A is required by the surrounding analyses. Hence the parser must employ top-down prediction to be sure that no constituent is lost. This in turn forces either left-to-right or right-to-left analysis, depending on whether the prediction for A took place at its left or at its right boundary.

In the case of a tabular parsing method, both the bidirectional and the top-down mono-directional requirements can be accomplished by defining an appropriate context-free grammar that covers the input grammar G . For the sake of simplicity, such a covering problem is discussed below under the hypothesis that G is expressed in a null production free form. Appendix A shows how to drop this restriction. The next definition introduces the nonterminals of the studied cover, that is, the symbols that will be used in representing partial analyses for productions of the input grammar.

Definition 9. Let $G = (N, \Sigma, P, S)$ be a context-free grammar. The *i-items* associated with G are all and only the elements defined as follows:

- (i) for every production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ in P :
 - (a) $I_r^{(s,t)}$ is an i-item, $0 \leq s < t \leq \pi_r$, $(s, t) \neq (0, \pi_r)$;
 - (b) $I_r^{(0,0)}$ and $I_r^{(\pi_r, \pi_r)}$ are i-items;
- (ii) for every $A \in N$, I_A is an i-item.

In the following, symbol $I_G^{(I)}$ will denote the set of all i-items. In the next definition we will cover an input grammar in such a way that the bidirectional requirement is met. Without loss of generality, we assume that only one production in G rewrites the start symbol S .

Definition 10. Let $G = (N, \Sigma, P, S)$ be a context-free grammar. The *island cover* (*i-cover* for short) $G_I = (I_G^{(I)}, \Sigma, P_I, I_S)$ for G is a context-free grammar whose productions P_I are partitioned into the following three sets:

- (i) for every production p_r in P , the *prediction set* $P_I^{(0)}$ contains the productions
 - $I_r^{(0,0)} \rightarrow \varepsilon$,
 - $I_r^{(\pi_r, \pi_r)} \rightarrow \varepsilon$;
- (ii) the *projection set* $P_I^{(1)}$ is defined as follows:
 - (a) for every production $D_r \rightarrow Z_{r,1}$ in P , the production $I_{D_r} \rightarrow X_I$ belongs to $P_I^{(1)}$, where $X_I = I_{Z_{r,1}}$ if $Z_{r,1} \in N$ and $X_I = Z_{r,1}$ if $Z_{r,1} \in \Sigma$;
 - (b) for every production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ in P , $\pi_r > 1$, and for every s , $1 \leq s \leq \pi_r$, the production $I_r^{(s-1,s)} \rightarrow X_{I,s}$ belongs to $P_I^{(1)}$, where $X_{I,s} = I_{Z_{r,s}}$ if $Z_{r,s} \in N$ and $X_{I,s} = Z_{r,s}$ if $Z_{r,s} \in \Sigma$;
- (iii) the *expansion set* $P_I^{(2)}$ is defined as follows. For every production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ in P , $\pi_r > 1$:
 - (a) for every i-item $I_r^{(s,t)}$ and for every u , $s < u < t$, the productions (when defined)
 - $I_r^{(s,t)} \rightarrow I_r^{(s,t-1)} X_{I,t}$,
 - $I_r^{(s,t)} \rightarrow Y_{I,s} I_r^{(s+1,t)}$,
 - $I_r^{(s,t)} \rightarrow I_r^{(s,u)} I_r^{(u,t)}$
 belong to $P_I^{(2)}$, where $X_{I,t} = I_{Z_{r,t}}$ if $Z_{r,t} \in N$, $X_{I,t} = Z_{r,t}$ if $Z_{r,t} \in \Sigma$ and $Y_{I,s} = I_{Z_{r,s+1}}$ if $Z_{r,s+1} \in N$, $Y_{I,s} = Z_{r,s+1}$ if $Z_{r,s+1} \in \Sigma$;
 - (b) for every u , $0 < u < \pi_r$, the productions
 - $I_{D_r} \rightarrow I_r^{(0,\pi_r-1)} X_I$,
 - $I_{D_r} \rightarrow Y_I I_r^{(1,\pi_r)}$,
 - $I_{D_r} \rightarrow I_r^{(0,u)} I_r^{(u,\pi_r)}$

belong to $P_I^{(2)}$, where conditions similar to the ones expressed in (iii)a hold for symbols $X_I, Y_I \in (I_G^{(I)} \cup \Sigma)$ with respect to Z_{I,π_r} and $Z_{r,1}$.

Example 11. We show how the definition of i-cover applies to grammar G_{cl} introduced in Example 1 (Section 2.3). The island cover associated with such a grammar contains the following productions:

$$\begin{aligned}
 P_I^{(0)}: & \quad I_1^{(0,0)} \rightarrow \varepsilon, & I_2^{(0,0)} \rightarrow \varepsilon, & I_3^{(0,0)} \rightarrow \varepsilon, \\
 & \quad I_1^{(2,2)} \rightarrow \varepsilon; & I_2^{(3,3)} \rightarrow \varepsilon; & I_3^{(2,2)} \rightarrow \varepsilon; \\
 P_I^{(1)}: & \quad I_1^{(0,1)} \rightarrow I_{NP}, & I_2^{(0,1)} \rightarrow cl, & I_3^{(0,1)} \rightarrow det, \\
 & \quad I_1^{(1,2)} \rightarrow I_{VP}; & I_2^{(1,2)} \rightarrow v, & I_3^{(1,2)} \rightarrow n; \\
 & & I_2^{(2,3)} \rightarrow I_{NP}; & \\
 P_I^{(2)}: & \quad I_1^{(0,1)} \rightarrow I_1^{(0,0)} I_{NP}, & I_2^{(0,1)} \rightarrow I_2^{(0,0)} cl, & I_3^{(0,1)} \rightarrow I_3^{(0,0)} det, \\
 & \quad I_1^{(1,2)} \rightarrow I_{VP} I_1^{(2,2)}; & I_2^{(2,3)} \rightarrow I_{NP} I_2^{(3,3)}; & I_3^{(1,2)} \rightarrow n I_3^{(2,2)}; \\
 I_S \rightarrow & \quad I_1^{(0,1)} I_{VP}, & I_2^{(0,2)} \rightarrow I_2^{(0,1)} v, & I_2^{(1,3)} \rightarrow I_2^{(1,2)} I_{NP}, \\
 I_S \rightarrow & \quad I_{NP} I_1^{(1,2)}, & I_2^{(0,2)} \rightarrow cl I_2^{(1,2)}, & I_2^{(1,3)} \rightarrow v I_2^{(2,3)}, \\
 I_S \rightarrow & \quad I_1^{(0,1)} I_1^{(1,2)}, & I_2^{(0,2)} \rightarrow I_2^{(0,1)} I_2^{(1,2)}, & I_2^{(1,3)} \rightarrow I_2^{(1,2)} I_2^{(2,3)}, \\
 & & I_{VP} \rightarrow I_2^{(0,2)} I_{NP}, & I_{NP} \rightarrow I_3^{(0,1)} n, \\
 & & I_{VP} \rightarrow cl I_2^{(1,3)}, & I_{NP} \rightarrow det I_3^{(1,2)}, \\
 & & I_{VP} \rightarrow I_2^{(0,1)} I_2^{(1,3)}, & I_{NP} \rightarrow I_3^{(0,1)} I_3^{(1,2)}, \\
 & & I_{VP} \rightarrow I_2^{(0,2)} I_2^{(2,3)}, &
 \end{aligned}$$

Unfortunately, the above cover is quite redundant: string $w = \text{det } n \text{ cl } v$ $\text{det } n$ is unambiguous in G_{cl} , but it has many distinguishable parse trees in the associated i-cover. Anticipating ideas developed in Section 4.3, let us assume that each production in $P_I^{(1)}$ can only be used when the constituent in its right-hand side dominates some island, and productions in $P_I^{(0)}$ can only be used in positions adjacent to constituents dominating some island. Then we get a considerable reduction in the ambiguity of the i-cover. Consider the case in which items 5 and 6 in w , corresponding to symbols v and det , are selected as islands. Fig. 6 presents one possible parse tree for such a case.

Note that G_I is a general bidirectional cover for G , in the sense that any symbol in G can be the triggering element for the productions whose right-hand side it belongs to (set $P_I^{(1)}$) and any expansion order is possible for such an element (set $P_I^{(2)}$), i.e., G_I does not promote one particular order. In addition, observe that set $P_I^{(0)}$ allows top-down prediction to the left as well as to the

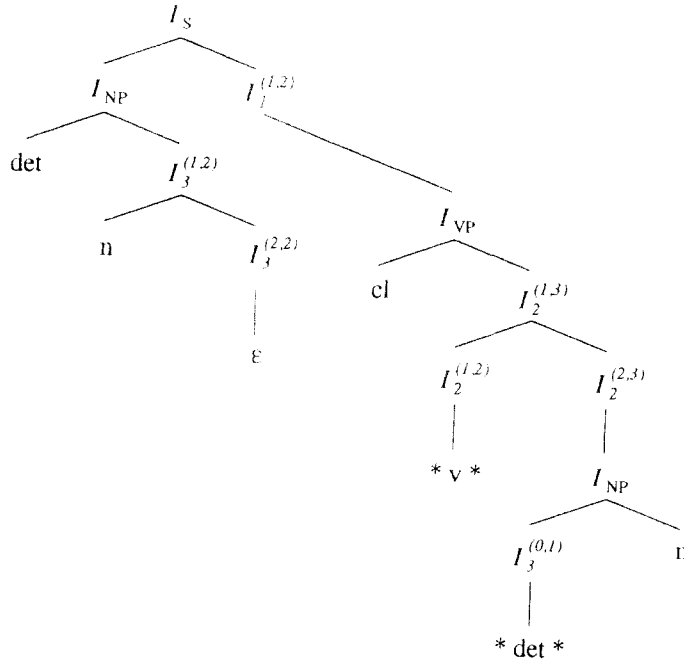


Fig. 6. A possible parse tree for sentence $w = \text{det n cl v det n}$ under the i-cover associated with G_{cl} . Island elements within w are marked with symbol '*'.

right for every production of the initial grammar G . We emphasize that the redundancy of i-covers is due to the unpredictable way in which islands can be selected within input strings. Much of the work in Section 4.3 is devoted to the development of a technique that eliminates redundancy in a dynamic way.

4.2. Parsing efficiency

In this section a computational perspective is taken on the problem of island-driven parsing. The definition of island cover is discussed at length along with its consequences for parsing efficiency. Furthermore, different proposals for island-driven parsing found in the literature are related to the class of island cover grammars.

Let $G_I = (I_G^{(I)}, \Sigma, P_I, I_S)$ be the island cover associated with a context-free grammar $G = (N, \Sigma, P, S)$. From the definition of i-item we can derive the equality $|I_G^{(I)}| = |N| + \frac{1}{2} \sum_{k=1}^{|P|} [\pi_k(\pi_k + 1) + 2]$, that is the number of nonterminals of G_I is $\Theta(\pi_G |G|)$ (recall that π_G is the maximum among the length of all right-hand sides in G). Observe also that, for every nonterminal $I_r^{(s,t)}$, a number $\Theta(t - s)$ of binary productions are found in $P_I^{(2)}$ with this symbol as the left-hand side. This should be compared with the head-driven case, where each nonterminal having the form $I_r^{(s,t)}$ can be rewritten in G_H in no more than two different ways, as already noted. It follows that there are

$\Theta(\pi_G^2|G|)$ productions in P_I , and therefore $|G_I| = \Theta(\pi_G^2|G|)$. As a consequence, in the worst case, a tabular method of the kind discussed in Section 2 will take at least an amount of time proportional to the quantity $\pi_G^2|G|n^3$ to recognize an input string of length n when working on G_I . For each entry $t_{i,j}$ of the recognition matrix, such a method has to recognize $\Theta(\pi_G|G|)$ nonterminals of the form $I_r^{(s,t)}$, and in the worst case each nonterminal can be inserted in $t_{i,j}$ in $\Theta(\pi_G(j-i))$ different ways.

As discussed in Section 3.2, fixed order expansion for partial analyses has been introduced into some bidirectional methods for head-driven parsing. This has the desired consequence of reducing analysis redundancy. As opposed to head-driven parsing, island-driven parsing is here conceived as a dynamic strategy that depends on the input string. Given that words to be chosen as starting islands can fall more than one in the same constituent in an unpredictable way, it is not clear how to accomplish island-driven strategies through static order expansion. The need for a more general parsing strategy, which allows for dynamic order expansion is the reason for the limiting result reported above.

A general parsing framework has been proposed in [25] for a class of grammars that is considerably larger than the class CFG. The authors apply their results to context-free grammars, obtaining an interesting bidirectional parsing technique that allows for dynamic island selection and dynamic expansion order. However such a technique cannot mimic in a direct way the island-driven strategy that has been defined in Section 4.1, due to the lack of any top-down capability. Most important here, in the cited work Pereira and Warren observe that their technique can support parsing strategies in which a single partial analysis of a production right-hand side does not correspond to a (contiguous) sub-string of the input string w . It should be observed that these strategies require a great deal of computational effort. In fact, in the general case time complexity can grow with a function in the length of the input string that is a polynomial of degree k , for any $k \geq 3$, and with an exponential in the length of the input context-free grammar.

The analysis of i-covers presented so far applies to cases of every-way ambiguous grammars. Note that it is rather unlikely for any practical application to meet such a worst case. However, some consequences of the i-cover definition are derived below that affect the average parsing performance as well.

First of all, note that the problem of analysis redundancy observed for head covers is inherited by island covers to an even greater extent. Intuitively this can be seen by observing that, in the case of island-driven parsing, not only do we deal with analysis proliferation due to the possibility of expanding partial analyses on two sides, but also with cases in which partial analyses collide, because more than one island can be selected within the same constituent in the input sentence. Let p_r be a production of the input grammar G . The cover G_I uses a number $\Theta(\pi_r^2)$ of nonterminals to “cover” the right-hand side of p_r , but every such a symbol can be obtained in a number of different ways

that grows with π_r , as already observed. It is found that, by using a tabular technique, the completion of the analysis for p_r may take $\Theta(\pi_r^3)$ steps (to be compared with the left-to-right case, where the same process takes $\Theta(\pi_r)$ steps).

The tabular recognizer for context-free languages presented in [4] can be seen as a bidirectional island-driven recognizer, where every symbol in the input string has been selected as an island. This restriction allows one to get rid of any top-down component in the recognition strategy, and the resulting algorithm behaves in a pure bottom-up fashion. Furthermore analysis proliferation is bounded in such a method by the additional constraint that the input context-free grammar must be cast in CNF, and therefore $\pi_r \leq 2$ for every production p_r . The tabular method presented in [7] is an extension to general context-free grammars of the method studied by Bossi et al. In such a method the authors face the problem of analysis redundancy by allowing partial analysis expansion to the left only and by disallowing island collision. This means that partial analyses are only expanded in a step by step fashion. In this way the completion of the analysis of the right-hand side of each production p_r is accomplished in $\Theta(\pi_r^2)$ steps. The worst case always holds because every symbol in the input string is regarded as an island, and therefore every element in the right-hand side of p_r triggers a partial analysis for p_r itself.

A second problem affects the efficiency of island-driven parsing in the average case. This problem originates from the interaction between island-driven analyses and top-down analyses, which must be carried out in order to recover constituents that fall apart from the selected islands. The overall result is that, in addition to the (redundant) island-outward recognition discussed above, every constituent gets analyzed twice more by top-down prediction at its boundaries. Although in this case the order of analysis redundancy is less than the one resulting from the bidirectional requirement, in practical cases it affects parsing performance as much as the latter, because in most natural language applications the maximum length of the production's right-hand side is not greater than three or four.

In the next section, a bidirectional method is presented that overcomes the analysis redundancy due to the two different problems discussed above. Such a method extends a technique that was originally employed in [30].

4.3. An island-driven tabular recognizer

This section presents a tabular method that recognizes languages $L(G)$, G being a context-free grammar in a null-production free form. The method adopts the island-driven parsing strategy defined in Section 4.1. Partial analyses are obtained starting from the selected islands within the input sentence and proceeding outward. The mixed bottom-up/top-down strategy adopted by the method should not be confused with the more familiar bottom-up strategies with top-down filtering found in the natural language parsing literature (see [46] for discussion). In the studied algorithm, some (not preselected)

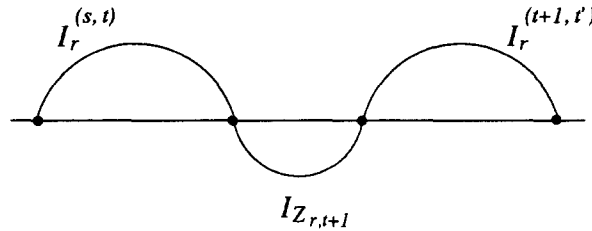


Fig. 7. Example of two partial analyses $I_r^{(s,t)}$ and $I_r^{(t+1,u)}$ competing for a common subconstituent $I_{Z_{r,t+1}}$ within the input string.

constituents are entirely analyzed in a top-down fashion, if they do not include any island.

The specification of the algorithm is based on the definition of the i-cover G_I associated with G . The two fundamental steps that are found within the method, called *left-expand* and *right-expand* steps, combine two adjacent analyses according to productions in $P_I^{(2)}$. Productions in $P_I^{(1)}$ are instead used by the *init* step and the *project* step, which respectively try to trigger analyses from the selected islands or from the already found constituents that contain at least one island. Furthermore, the *predict* step uses productions in $P_I^{(0)}$ in making top-down predictions for the requested constituents. Apart from the *init* step, the execution of the remaining steps on an i-item inserted in the recognition matrix T can be done in any order and there is no restriction at all on the order in which i-items must be processed. Finally, a procedure called *synchronize* has been included in the definition of the algorithm. Such a procedure solves cases of conflict between left-to-right and right-to-left top-down analyses, as will be discussed at length below.

Before presenting the specification of the studied algorithm, we informally discuss the technique that has been adopted in order to solve the analysis proliferation problem due to the bidirectional expansion requirement and to partial analysis collision. In the following, we will refer to i-items of the form $I_r^{(s,t)}$ as *partial* analyses, while i-items having the form I_A will be called *complete* analyses. Two partial analyses $I_r^{(s,t)}$ and $I_r^{(t+1,u)}$ found from left to right within the input string are said to be *competing* for a common subconstituent if a complete analysis $I_{Z_{r,t+1}}$ is found that exactly spans the portion of the input between $I_r^{(s,t)}$ and $I_r^{(t+1,u)}$ (see Fig. 7). Note that this is only possible if more than one island has been selected within the portion of the input string derived by the right-hand side of production p_r .

The subsumption blocking technique discussed in Section 3.3 can be employed in island-driven recognition as well to obviate cases of both side expansion of a partial analysis. Since in island-driven recognition we have the additional problem of island collision, we would like to generalize subsumption blocking in order to deal with such cases. Island collision begins with two partial analyses that are competing for a common subconstituent, i.e., a

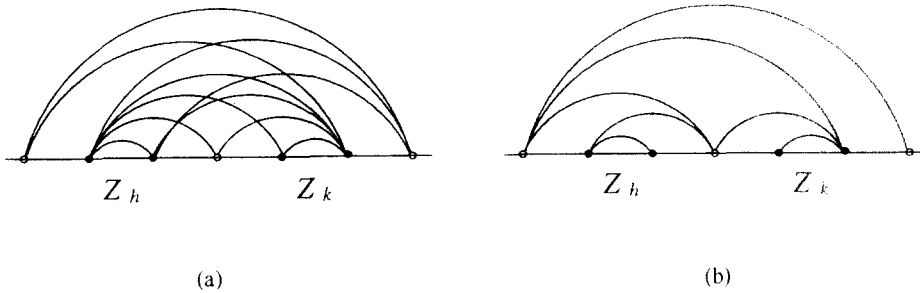


Fig. 8. Comparison between two cases of bidirectional island-driven parsing in the recognition of some production right-hand side. Two islands have been selected within the portion of the input string spanned by the right-hand side; subconstituent sharing between competing partial analyses has not been allowed in the case depicted in (b). Note that the subsumption blocking technique has been applied in both cases.

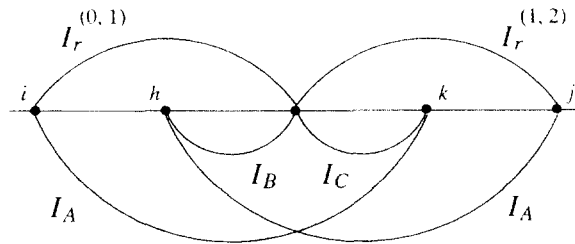


Fig. 9. Conflict between left-to-right and right-to-left top-down competing partial analyses.

complete analysis. As shown in Fig. 8, if we allow free sharing of complete analyses among competing partial analyses, we will introduce redundant partial analyses, even if we apply the subsumption blocking technique. (This also applies to the similar case in which two partial analyses are competing for a common terminal symbol.)

The most straightforward generalization of the subsumption blocking technique to cases of complete analysis sharing does not work. Assume we just state that whenever some constituent $I_{Z_{r,t+1}}$ has been paired with a partial analysis, say $I_r^{(s,t)}$ to its left, producing a partial analysis $I_r^{(s,t+1)}$, it cannot be paired with any other competing partial analysis $I_r^{(t+1,u)}$ to its right. Then we will have some undesired consequences in cases of interaction between left-to-right and right-to-left top-down analyses, as the following example shows.

In Fig. 9 no island has been selected in the portion of the input string represented by the substring $a_{i+1} \cdots a_j$. Assume production $A \rightarrow BC$ to be the r th production in P . Then productions $I_A \rightarrow I_r^{(0,1)} I_C$ and $I_A \rightarrow I_B I_r^{(1,2)}$ belong to $P_I^{(2)}$. Assume also that top-down recognition has revealed complete analyses I_B and I_C along with partial analyses $I_r^{(0,1)}$ and $I_r^{(1,2)}$, with the extensions depicted in the figure. If at this point I_C is paired at its left with $I_r^{(0,1)}$, to obtain constituent I_A , then our extension of the subsumption blocking

technique will prevent any top-down prediction $I_r^{(2,2)}$ at position k from using constituent I_C , because $I_r^{(2,2)}$ is a competing analysis with respect to $I_r^{(0,1)}$. Symmetrically, if I_B is paired at its right with the partial analysis $I_r^{(1,2)}$, then any top-down prediction $I_r^{(0,0)}$ at position h will be prevented from using constituent I_B at its right, because $I_r^{(0,0)}$ is a competing analysis with respect to $I_r^{(1,2)}$. As a consequence, no top-down recognition for constituent I_A from h to k can ever take place. Such a constituent would then be lost, because of the absence of any island within the portion of the string under analysis.

The island-driven algorithm we present here adopts the following solution for the discussed problem.² We need to establish some sort of synchronization between left-to-right and right-to-left top-down analyses, still guaranteeing the completeness of the algorithm. Assume that we arbitrarily break into two substrings each portion of the input string lying between two consecutive islands. These strings will be called *left* and *right substrings*. (The substring to the left of the leftmost island will be a right substring and the substring to the right of the rightmost island will be a left substring.) Then we impose the following restrictions on the recognition process:

- every complete analysis entirely contained within a left substring can only be paired with a partial analysis adjacent to its left; at the same time, every complete analysis entirely contained within a right substring can only be paired with a partial analysis adjacent to its right;
- once a complete analysis that does not dominate any island and that includes the boundary between a left and a right substring is paired with a partial analysis at some side, then it cannot be paired with any other *competing* partial analysis at the opposite side;
- every complete analysis that dominates some island cannot be paired with *any* partial analysis;
- within a left substring, only left-to-right top-down predictions can be made by the algorithm, while right-to-left top-down predictions are allowed within right substrings only.

As a result of the interaction among the four constraints above, we have that in a left substring only left-to-right top-down recognition can take place, and a symmetrical condition is found for right substrings. Moreover, at each boundary between a left and a right substring, an appropriate “synchronization” takes place between left and right analyses. An example will be of some help in understanding how these constraints interact.

In Fig. 10 we have depicted a portion of the input string between two consecutive islands (therefore the complete analysis I_{A_1} does not include any island). Position i represents the (arbitrary) boundary between the left substring and the right substring. Assume that constituent I_{A_1} is rewritten by cover G_I accord-

² A simpler solution has been reported in previous work by these authors ([32]). Unfortunately it is actually not the case that an algorithm based on such a solution is able correctly to recognize an input string if i-items inserted in the recognition matrix T are processed in any possible order, as was verified later on.

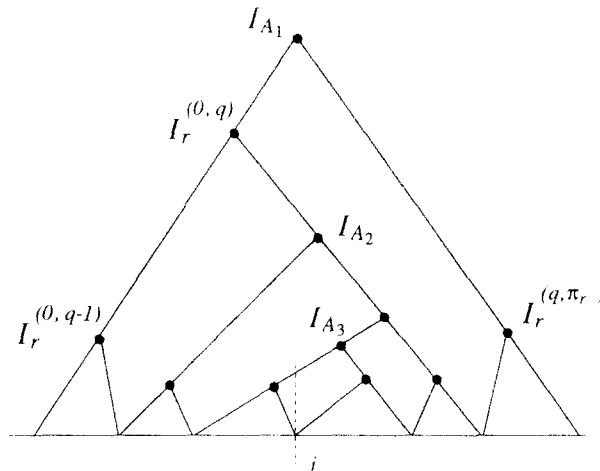


Fig. 10. Interaction between left-to-right and right-to-left partial analyses in a portion of the input string between two adjacent islands. No island is dominated by symbol I_{A_1} .

ing to a set of productions that correspond to production $r: A_1 \rightarrow Z_{r,1} \cdots Z_{r,\pi_r}$ of the input grammar G . Assume also that constituent $Z_{r,q} = A_2$ dominates position i for some q , $1 \leq q \leq \pi_r$. We have therefore a set of left children of A_1 , $\{Z_{r,1}, \dots, Z_{r,q-1}\}$, and a set of right children of A_1 , $\{Z_{r,q+1}, \dots, Z_{r,\pi_r}\}$. Then, under the first constraint above, our algorithm collects the left children in a left-to-right fashion and the right children in a right-to-left fashion. Nonterminals $I_r^{(0,q-1)}$ and $I_r^{(q,\pi_r)}$ in the figure are the results of these processes. This analysis iterates for constituent I_{A_2} , until we find a constituent whose children are each entirely contained within the left or the right substrings (constituent I_{A_3} in the figure). Then our second constraint assures us that constituents I_{A_3} and I_{A_2} will not be shared between two competing partial analyses, i.e., the partial analyses for the left and the right siblings of these constituents. Finally, the third and the last constraints above prevent the algorithm from inserting redundant symbols in the recognition matrix. In fact, any complete analysis that dominates some island will trigger by itself the analyses for the right-hand side of the relevant productions, while top-down left-to-right (right-to-left) predictions are not useful within a right (left) substring.

If the subsumption blocking technique and the discussed restrictions are used by a tabular algorithm that works on cover G_I , then the construction of redundant analyses and the formation of conflicts of the kind shown in Fig. 9 is completely prevented. A precise formulation of this statement is reported in Appendix C. The studied algorithm implements the subsumption blocking technique and the four constraints above by manipulating set valued symbols $Q(i, j, X_I, d)$, where $0 \leq i \leq j \leq |w|$ (w the input string), $X_I \in I_G^{(I)} \cup \Sigma$ and $d \in \{\text{left}, \text{right}\}$. For every $r \in \{1..|P|\}$ and $s \in \{1..\pi_r\}$, the insertion of pair (r, s) in $Q(i, j, X_I, d)$ prevents symbol X_I from i to j from ever being paired

at side $d = \text{left}$ (respectively, $d = \text{right}$) with a partial analysis $I_r^{(s',s)}$ or with a complete analysis $I_{Z_{r,s}}$ (respectively, $I_r^{(s,s'')}$ or $I_{Z_{r,s+1}}$). More specifically, the subsumption blocking technique along with the second of the four constraints discussed above are implemented by the already mentioned left-/right-expand steps. The fourth constraint is implemented by the predict step and the two remaining constraints are implemented within the synchronize procedure. As shown in Appendix C, the discussed solution to the redundancy problem has the interesting property of filtering island-outward analyses by requesting a sort of “compatibility” with the surrounding islands.

We turn now to a formal specification of the studied algorithm. Let $G_I = (I_G^{(I)}, \Sigma, P_I, I_S)$ be an i-cover for $G = (N, \Sigma, P, S)$. For notational convenience, the two following functions will be used. Let $A_I \rightarrow X_I Y_I$ be a production in $P_I^{(2)}$. If $X_I = I_r^{(s,t)}$ then $\text{mid}(A_I \rightarrow X_I Y_I) = t$, if $Y_I = I_r^{(s,t)}$ then $\text{mid}(A_I \rightarrow X_I Y_I) = s$ (function mid is well-defined by the construction of $P_I^{(2)}$). Let $A_I \rightarrow \alpha$ be a production in P_I obtained from a production p_r in P . If $A_I = I_r^{(s,t)}$ then $\text{index}(A_I \rightarrow \alpha) = (r, s, t)$; by extension, if $A_I = I_{D_r}$ then $\text{index}(A_I \rightarrow \alpha) = (r, 0, \pi_r)$. Finally, we will use a predicate *complete* defined on $I_G^{(I)} \cup \Sigma$ in such a way that *complete*(X_I) holds if and only if symbol X_I has the form I_{D_r} , for some r in $\{1..|P|\}$, or $X_I \in \Sigma$. As a convention, each set-valued symbol is initially assigned the empty set.

The algorithm is reported below. (The algorithm is not specified in its optimal form. To make it more readable, some obvious optimizations have been omitted.)

Algorithm 12.

Input. An i-cover $G_I = (I_G^{(I)}, \Sigma, P_I, I_S)$ for a context-free grammar G (with no null-production); a string $w = a_1 \cdots a_n$ in Σ^* , $n > 0$; integers M, i_k, p_k, q_k , $1 \leq M \leq n$ and $1 \leq k \leq M$, such that $q_{k-1} = p_k < i_k \leq q_k = p_{k+1}$ ($q_0 = 0, p_{M+1} = n$).

Output. Accept/reject.

Method. An $(n+1) \times (n+1)$ matrix T is used, whose entries take values in $\mathcal{P}(I_G^{(I)})$; symbols $Q(i, j, X_I, d)$ are also used, where $0 \leq i \leq j \leq n$, $X_I \in I_G^{(I)} \cup \Sigma$, $d \in \{\text{left}, \text{right}\}$; these symbols are assigned sets of elements in $\{1..|P|\} \times \{1..\pi_G\}$.

for every $i \in \{1..n\}$ do

comment: init step

synchronize($a_i, i-1, i$)

if $i = i_h$ for some h then

for every production $A_I \rightarrow a_i$ do

$t_{i-1,i} := t_{i-1,i} \cup \{A_I\}$

endfor

endif

```

endfor
for every  $A_I$  added to  $t_{i,j}$ ,  $0 \leq i \leq j \leq n$ , do
  synchronize( $A_I, i, j$ )
  comment: project step
  if  $i < i_h \leq j$  for some  $h$  then
    for every production  $B_I \rightarrow A_I$  do
       $t_{i,j} := t_{i,j} \cup \{B_I\}$ 
    endfor
  endif
  comment: predict step
  if  $p_h < i < i_h$  for some  $h$ , not complete( $A_I$ ), then
    l-predict( $A_I, i$ )
  endif
  if  $i_h \leq j < q_h$  for some  $h$ , not complete( $A_I$ ), then
    r-predict( $A_I, j$ )
  endif
  comment: left-expand step
  for every production  $B_I \rightarrow X_I A_I$  do
     $(r, s, t) := \text{index}(B_I \rightarrow X_I A_I)$ 
     $z := \text{mid}(B_I \rightarrow X_I A_I)$ 
    if  $(r, z) \notin Q(i, j, A_I, \text{left})$  then
      for every  $i' \leq i$  do
        if  $(r, z) \notin Q(i', i, X_I, \text{right})$ ,  $(X_I \in t_{i',i}$  or  $X_I = a_i, i' = i - 1)$ , then
           $t_{i',j} := t_{i',j} \cup \{B_I\}$ 
           $Q(i', i, X_I, \text{left}) := Q(i', i, X_I, \text{left}) \cup \{(r, s)\}$ 
           $Q(i, j, A_I, \text{right}) := Q(i, j, A_I, \text{right}) \cup \{(r, t)\}$ 
        endif
      endfor
    endif
  endfor
endif
endfor
comment: right-expand step
for every production  $B_I \rightarrow A_I X_I$  do
   $(r, s, t) := \text{index}(B_I \rightarrow A_I X_I)$ 
   $z := \text{mid}(B_I \rightarrow A_I X_I)$ 
  if  $(r, z) \notin Q(i, j, A_I, \text{right})$  then
    for every  $j' \geq j$  do
      if  $(r, z) \notin Q(j, j', X_I, \text{left})$ ,  $(X_I \in t_{j,j'}$  or  $X_I = a_{j+1}, j' = j + 1)$ , then
         $t_{i,j'} := t_{i,j'} \cup \{B_I\}$ 
         $Q(i, j, A_I, \text{left}) := Q(i, j, A_I, \text{left}) \cup \{(r, s)\}$ 
         $Q(j, j', X_I, \text{right}) := Q(j, j', X_I, \text{right}) \cup \{(r, t)\}$ 
      endif
    endfor
  endif
endfor
endfor
endfor

```

if $I_S \in t_{0,n}$ then accept else reject endif
end.

Procedure *synchronize* employed by Algorithm 12 implements the first and the third of the four constraints previously discussed, in order to prevent analysis redundancy in cases of island collision. The procedure is straightforward and is reported in the following:

```

procedure synchronize( $X_I, i, j$ )
begin
  if complete( $X_I$ ) then
    if  $i < i_h \leq j$  for some  $h$  then
      for every production  $A_I \rightarrow X_I$  do
        ( $r, s, t$ ) := index( $A_I \rightarrow X_I$ )
         $Q(i, j, X_I, left) := Q(i, j, X_I, left) \cup \{(r, s)\}$ 
         $Q(i, j, X_I, right) := Q(i, j, X_I, right) \cup \{(r, t)\}$ 
      endfor
    endif
    if  $p_h \leq i < j < i_h$  for some  $h$  then
      for every production  $A_I \rightarrow X_I$  do
        ( $r, s, t$ ) := index( $A_I \rightarrow X_I$ )
         $Q(i, j, X_I, left) := Q(i, j, X_I, left) \cup \{(r, s)\}$ 
      endfor
    endif
    if  $i_h \leq i < j \leq q_h$  for some  $h$  then
      for every production  $A_I \rightarrow X_I$  do
        ( $r, s, t$ ) := index( $A_I \rightarrow X_I$ )
         $Q(i, j, X_I, right) := Q(i, j, X_I, right) \cup \{(r, t)\}$ 
      endfor
    endif
  endif
end.

```

Finally, procedures *l-predict* and *r-predict* used by Algorithm 12 must be implemented in such a way that prediction of the same null productions by different *i*-items is avoided. Again, this is done by extra bookkeeping. Define $left(I_r^{(s,t)}) = (A_I, P_l)$, where $P_l = \{I_u^{(\pi_u, \pi_u)} \rightarrow \varepsilon \mid s > 0, Z_{r,s} = D_u\}$ and if P_l is not empty, $A_I = I_{D_u}$ for $I_u^{(\pi_u, \pi_u)} \rightarrow \varepsilon$ in P_l , otherwise A_I is undefined. P_l is the subset of null productions of G_I that can start recognition of nonterminal A_I from its right, where A_I is needed to extend partial analysis $I_r^{(s,t)}$ to the left. Nonterminal A_I will be used to block the search for productions that have already been predicted at the same position within the input string by other partial analyses. In a complete symmetrical way, we define $right(I_r^{(s,t)}) = (A_I, P_r)$, such that $P_r = \{I_u^{(0,0)} \rightarrow \varepsilon \mid t < \pi_r, Z_{r,t+1} = D_u\}$ and if P_r is not empty, $A_I = I_{D_u}$ for $I_u^{(0,0)} \rightarrow \varepsilon$ in P_r , otherwise A_I is undefined. We will also

use symbols $S(i, d)$ for $1 \leq i \leq n$ and $d \in \{left, right\}$: sets of nonterminals of G_I having the form I_A are assigned to these symbols. Procedures l-predict and r-predict implement a recursive search as reported in the following:

```

procedure l-predict( $A_I, i$ )
begin
  ( $B_I, P_l$ ) := left( $A_I$ )
  if  $B_I \notin S(i, left)$  then
     $S(i, left) := S(i, left) \cup \{B_I\}$ 
    for every production  $I_u^{(\pi_u, \pi_u)} \rightarrow \varepsilon$  in  $P_l$  do
       $t_{i,i} := t_{i,i} \cup \{I_u^{(\pi_u, \pi_u)}\}$ 
      l-predict( $I_u^{(\pi_u, \pi_u)}, i$ )
    endfor
  endif
end.

```

```

procedure r-predict( $A_I, j$ )
begin
  ( $B_I, P_r$ ) := right( $A_I$ )
  if  $B_I \notin S(j, right)$  then
     $S(j, right) := S(j, right) \cup \{B_I\}$ 
    for every production  $I_u^{(0,0)} \rightarrow \varepsilon$  in  $P_r$  do
       $t_{j,j} := t_{j,j} \cup \{I_u^{(0,0)}\}$ 
      r-predict( $I_u^{(0,0)}, j$ )
    endfor
  endif
end.

```

Algorithm 12 stops when no further i-item can be added to the recognition matrix T . A characterization of the set of all i-items inserted in T by the method is reported in Appendix C; from such an invariant, the correctness of the algorithm can be obtained directly. To conclude the present section, computational complexity issues are addressed. The following result shows that the upper bound $O(\pi_G^2 |G| |w|^3)$ for island-driven recognition discussed in Section 4.2 is actually met by cover G_I .

Theorem 13. *Let $f(G_I, w)$ be the running time of Algorithm 12. Then $f(G_I, w) = O(\pi_G^2 |G| |w|^3)$.*

Proof. We consider membership, insertion and functions *index*, *mid*, *complete*, *left*, *right* to be elementary operations that can be carried out in constant time.

Note that the number of productions in $P_I^{(1)} \cup P_I^{(0)}$ is bounded by $|G|$. For given i and j , procedures synchronize, l-predict and r-predict along with the init step, the predict step and the project step never visit more than once a

production in $P_I^{(1)} \cup P_I^{(0)}$. Then the amount of time required by all possible executions of these steps is bounded by $O(|G||w|^2)$.

We are left with the two more complex steps of the algorithm, namely the left-expand and the right-expand steps. We recall that $|P_I^{(2)}| = O(\pi_G^2|G|)$. In the case of the left-expand step, for given i' , i and j no production in $P_I^{(2)}$ is visited more than once. Then the overall amount of time charged to this step is bounded by $O(\pi_G^2|G||w|^3)$. The same reasoning also applies to the right-expand step, and this proves the theorem. $\square$

The result above is, up to our knowledge, the best time upper bound that has been attested for general island-driven recognition of context-free languages. The bound is slightly worse than the head-driven case. This is a counterpart to the increased flexibility of the island-driven strategy. As already discussed, in island-driven recognition the worst-case dependency on the size of G reported above can also be found in the average case. The subsumption blocking prescription avoids this problem by eliminating analysis redundancy. In this way, any constituent D_r found within the input string will be constructed in no more than π_r steps by Algorithm 12, starting from an immediate realization of the right-hand side $Z_{r,1}Z_{r,2} \cdots Z_{r,\pi_r}$. In practical cases, the resulting performance is comparable to the one of standard mono-directional methods, but, due to its flexibility, the algorithm can be also used in cases where mono-directional strategies are completely inadequate.

5. Conclusions

For natural language parsing, devices that can process sentences in a bidirectional fashion are very attractive. Theoretical motivations concerning information distribution in natural language as well as application related arguments have been presented in favor of this claim. To provide a general treatment, we have introduced a formal framework for dealing with bidirectional tabular parsing of general context-free languages. The framework, based on the idea of covering grammars, yields an appropriate level of abstraction for comparing the mono-directional and the bidirectional paradigms. We have shown a trade-off between flexibility and worst-case time complexity. The problem has already been observed from an intuitive point of view in the literature and has been ascribed to analysis redundancy. By exploiting our framework, we have been able to provide a detailed characterization of the problem.

We then proceeded to study a general technique which allows one to improve, on average, the performance of bidirectional tabular parsers, still maintaining the flexibility of bidirectional strategies. The technique has been exploited in the development of two parsing algorithms. The first algorithm is a head-driven bidirectional parser working under the hypothesis that the input grammar comes along with a specification of head elements found within

each production. We have also studied an algorithm for general island-driven parsing. The algorithm allows analyses to start from any dynamically chosen positions within the input sentence, and combines without restrictions bottom-up and top-down recognition. In this case also, the key problem of avoiding analysis redundancy has been solved in full generality. As already mentioned, the proposed algorithms have been developed with the aim of yielding results to be used in natural language processing. The head-driven algorithm can be successfully employed in parsing of “mixed order” languages, where heads of different categories subcategorize for complements in different directions. More generally, this algorithm can be used in all cases in which information is not uniformly distributed among words in the input. The island-driven algorithm can be primarily used for speech or hand-written input parsing, or in general for those cases where the input string may be corrupted or of such difficult analysis that usual left-to-right methods seem inadequate.

The proposed treatment of context-free bidirectional parsing should also be discussed in light of its possible extensions to formalisms of broad interest to the computational linguistics community, that are more powerful than the class of context-free grammars. In recent years, unification-based grammar formalisms that rest on an underlying context-free grammar have been widely employed in natural language processing applications (see [36] for a review). Provided these formalisms are order-free, that is they do not depend on the order in which immediate constituents are collected, our approach can easily be extended to handle them, following proposals in [35] for the mono-directional case. Another interesting class of rewriting systems that has received considerable attention in the computational linguistics community is tree adjoining grammar [17]. Although from a computational point of view tree adjoining grammars differ substantially from context-free grammars, our approach to bidirectional parsing has still proven successful and a bidirectional tabular parser for tree adjoining languages has been described in [21] following the proposals in this paper.

From a wider perspective, a number of other interesting applications suggest themselves. The bottom-up/top-down mixed strategy proposed for island-driven recognition can be of some relevance for the design of text generation algorithms, where commonly used techniques are derived from pure bottom-up bidirectional parsing methods (see for example [44]). Parallel application should also be considered, with the aim of improving average case performance of already known pure bottom-up parallel algorithms (see for instance [24,7]). We conclude by pointing out that many other applications call for non-mono-directional parsing, as for instance error correcting parsing, stochastic (score-based) parsing, syntactic pattern matching and graphic language parsing. We feel that all these areas could also benefit from the results presented here, and leave further investigation for future work.

Appendix A. Null productions

In Sections 3 and 4 it was assumed that the input context-free grammar does not contain null productions; in this appendix we show how to deal with this kind of productions, therefore extending previous results to the general case. We study in detail head covers; a symmetrical treatment can be devised for island covers as well.

If a reduced CFG $G = (N, \Sigma, P, S)$ contains null productions, we have that set $\mathcal{N}_G = \{A \in N \mid A \xrightarrow{*} \varepsilon\}$ is not empty. In principle, a head-driven tabular method that works with a head cover could interpret symbol ε as a particular kind of head element, adding then set $\mathcal{N}_H = \{I_A \mid A \in \mathcal{N}_G\}$ to each entry $t_{i,i}$, $0 \leq i \leq n$, in the recognition matrix T . This does not worsen the complexity results discussed in Section 3. More interestingly, we study in the following a cover for general form context-free grammars, that can be employed in a more efficient way by the head-driven recognizer defined by Algorithm 7. The cover is derived under the assumption that head elements in the input grammar do not belong to $\mathcal{N}_G$. On the other hand, if some head element Z_{r,τ_r} belongs to $\mathcal{N}_G$, then a pure bottom-up method like Algorithm 7 is unable to efficiently recognize the corresponding production $D_r \rightarrow Z_{r,1} \cdots Z_{r,\tau_r}$, because of the lack of the triggering condition. In these cases, efficient parsing can be obtained by using the more general island-driven strategy discussed in Section 4.

The following cover is defined by slightly adapting the head cover definition.

Definition 14. Let $G_H = (I_G^{(H)}, \Sigma, P_H, I_S)$ be the h-cover obtained from G without considering null productions in P . The cover $G_H^{(\varepsilon)} = (I_G^{(H)}, \Sigma, P_H^{(\varepsilon)}, I_S)$ for G is the extension of G_H specified as follows:

- (i) every production in P_H belongs to $P_H^{(\varepsilon)}$;
- (ii) for every production $A_H \rightarrow I_r^{(s,t)} I_D$ or $A_H \rightarrow I_D I_r^{(s,t)}$ in P_H , such that $I_D \in \mathcal{N}_H$, the production

$$A_H \rightarrow I_r^{(s,t)}$$
 belongs to $P_H^{(\varepsilon)}$.

Set $\mathcal{N}_G$ (and therefore set $\mathcal{N}_H$) can be computed from G using standard procedures; see for instance [15]. Note that in general, cover $G_H^{(\varepsilon)}$ will not be a reduced context-free grammar: again, useless productions can be eliminated by standard techniques. Algorithm 7 can easily be adapted to work on cover $G_H^{(\varepsilon)}$: it suffices to include the subsumption blocking in the project step, in order to deal with the new chain productions that have been added to $P_H^{(\varepsilon)}$. Most important, Algorithm 7 can be run on $G_H^{(\varepsilon)}$ with the same worst-case time complexity of Theorem 8; this follows from the fact that $|P_H^{(\varepsilon)}| < 2|P_H|$.

Cover $G_H^{(\varepsilon)}$ crucially makes use of chain productions in order to “skip over” those nonterminals that derive the null string, in a step by step fashion. It is

interesting to observe that, if we transform $G_H^{(e)}$ in order to eliminate chain productions from it, we obtain a cover corresponding to a technique that was originally presented in [13] to improve the performance of the Earley algorithm and that is quite often used in handling null productions. As far as the construction of such a cover is considered, no transformation is known for the elimination of chain productions that expands the input grammar linearly [39]: using known methods, in the case at hand the resulting cover will have a number of productions $O(\pi_G^2|G|)$, which will worsen the performance of the studied method.

Appendix B. Invariant for Algorithm 7

In Section 3.3 a head-driven tabular recognizer was reported (Algorithm 7); a characterization of the set of all h-items inserted in the recognition matrix by such a method is presented in the following, from which a correctness result can easily be obtained. A formal proof of the invariant below is outside the scope of the paper; nevertheless, the discussion of such a property will give us a better understanding of the proposed method.

To express the intuitive notion that a nonterminal of cover G_H represents a partial analysis that can further be extended at some side, we introduce a (reflexive) partial order relation S_H defined on $I_G^{(H)}$ by means of the following condition (universal quantification is intended on r, s and t):

$$S_H = \{ (I_r^{(s,t)}, I_r^{(s-h,t)}), (I_r^{(s,t)}, I_r^{(s,t+k)}) \mid h, k \geq 0 \} \cup \{ (I_r^{(0,t)}, I_{D_r}), (I_r^{(s,\pi_r)}, I_{D_r}) \} \cup \{ (I_{D_r}, I_{D_r}) \}. \quad (\text{B.1})$$

The invariant satisfied by Algorithm 7 is expressed in the following theorem.

Theorem 15. *Let G_H and w be defined as for the input of Algorithm 7. For every $A \in I_G^{(H)}$ and for every i and j , $0 \leq i < j \leq n$, the following conditions hold:*

- (i) *if $A \in t_{i,j}$ then $A \xrightarrow{*}_{G_H} a_{i+1} \cdots a_j$;*
- (ii) *if $A \xrightarrow{*}_{G_H} a_{i+1} \cdots a_j$ then there exists $B \in I_G^{(H)}$ such that $(A, B) \in S_H$ and either $B \in t_{i-d,j}$ or $B \in t_{i,j+d}$ for some integer $d \geq 0$.*

The invariant above should be compared with the general condition reported in (1) (Section 2.1). First of all, note that $R_T \equiv \text{true}$: this expresses the pure bottom-up behaviour of the recognizer. The valid prefix property mentioned in Section 2.3 for the Earley algorithm is lost here. This seems to be a limitation of the head-driven paradigm: since analysis of each constituent is started from its head, adjacency of two constituents is not guaranteed and can only be checked at the very end of their construction. Nevertheless, if we combine head-driven recognition with top-down prediction as proposed in [38] (see the discussion in Section 3.3) we obtain that a partial analysis $A \in I_G^{(H)}$ for an

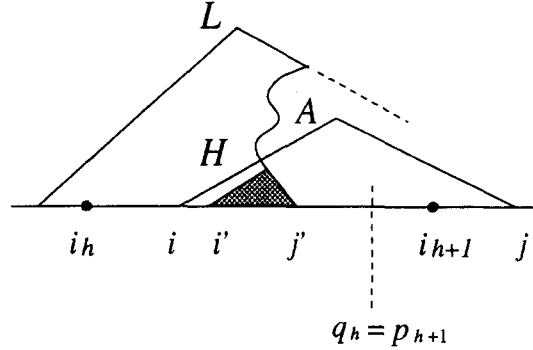


Fig. C.1. Condition (i) in Definition 16. We assume $i_{h+1} \leq j$.

argument of a head X is found by the recognizer only in case that somewhere else a partial analysis including X itself has been found: this applies recursively and the resulting property could be called “infix” property, in analogy with the left-to-right case. As a second point, note that the subsumption blocking technique discussed in Section 3.3 does not guarantee that the corresponding of the “if” statement in Condition (1) exactly holds. This has a graphical interpretation in the diagram of Fig. 3 in Section 3.2: depending on the order in which constituents surrounding the head are collected, it might be that the recognizer chooses a path not traversing partial analysis $A \in I_G^{(H)}$, although subsuming it in some way. Finally, note that the head-driven behaviour of the recognizer is not apparent in Theorem 15: it is implicitly stated in the definition of the cover G_H .

Appendix C. Invariant for Algorithm 12

An invariant property of the island-driven recognizer presented in Section 4.3 (Algorithm 12) is discussed in the following, with the aim of having more insight into the studied method. We assume that string w and integers $M, i_k, p_k, q_k, 1 \leq k \leq M$, are defined as for the input of Algorithm 12. Before presenting the result, we need to introduce additional notation.

Similarly to the head-driven case, we use a partial order relation defined on $I_G^{(I)}$ (universal quantification on indices r, s and t is intended):

$$\mathcal{S}_I = \{(I_r^{(s,t)}, I_r^{(s-h,t)}), (I_r^{(s,t)}, I_r^{(s,t+k)}) \mid h, k \geq 0\} \cup \{(I_r^{(0,t)}, I_{D_r}), (I_r^{(s,\pi_r)}, I_{D_r})\} \cup \{(I_{D_r}, I_{D_r})\}. \quad (\text{C.1})$$

The next definition introduces a property that expresses the “compatibility” of a constituent found within w with the surrounding islands. (Conditions (i) and (iii) below have been depicted in Figs. C.1 and C.2; Condition (ii) is symmetrical with respect to Condition (i).)

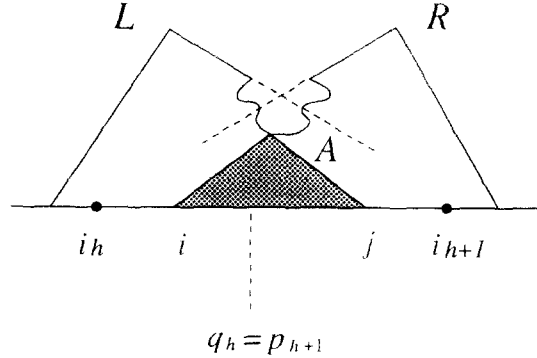


Fig. C.2. Condition (iii) in Definition 16.

Definition 16. Let ρ be a derivation of the form $A \xrightarrow{*}_{G_I} a_{i+1} \cdots a_j$, $A \in I_G^{(I)}$ and $0 \leq i \leq j \leq n$; $R_w^{(I)}(\rho)$ holds if and only if the following conditions hold:

- (i) if there exist h, i', j', H such that $i_h \leq i < i_{h+1}$, $i \leq i' \leq j' \leq \min\{j, q_h\}$ and such that ρ can be broken into $A \xrightarrow{*}_{G_I} a_{i+1} \cdots a_{i'} H a_{j'+1} \cdots a_j$ and $H \xrightarrow{*}_{G_I} a_{i'+1} \cdots a_{j'}$, then H has the form $I_r^{(0,t)}$ or I_{D_r} for some $1 \leq r \leq |P|$ and $L \xrightarrow{*}_{G_I} a_{i''+1} \cdots a_{i'} H \gamma$ for some $i'' < i_h$, $L \in I_G^{(I)}$, $\gamma \in (I_G^{(I)} \cup \Sigma)^*$;
- (ii) if there exist h, i', j', H such that $i_{h-1} \leq j < i_h$, $\max\{i, p_h\} \leq i' \leq j' \leq j$ and such that ρ can be broken into $A \xrightarrow{*}_{G_I} a_{i+1} \cdots a_{i'} H a_{j'+1} \cdots a_j$ and $H \xrightarrow{*}_{G_I} a_{i'+1} \cdots a_{j'}$, then H has the form $I_r^{(s, \pi_r)}$ or I_{D_r} for some $1 \leq r \leq |P|$ and $R \xrightarrow{*}_{G_I} \gamma H a_{j''+1} \cdots a_{j'}$ for some $j'' \geq i_h$, $R \in I_G^{(I)}$, $\gamma \in (I_G^{(I)} \cup \Sigma)^*$;
- (iii) if there exists h such that $i_h \leq i < q_h < j < i_{h+1}$, then either A has the form $I_r^{(0,s)}$ or I_{D_r} and $L \xrightarrow{*}_{G_I} a_{i'} \cdots a_i A \gamma$, or A has the form $I_r^{(s, \pi_r)}$ or I_{D_r} and $R \xrightarrow{*}_{G_I} \gamma' A a_{j+1} \cdots a_{j'}$, for some $1 \leq r \leq |P|$, $i' < i_h$, $j' \geq i_{h+1}$, $L, R \in I_G^{(I)}$, $\gamma, \gamma' \in (I_G^{(I)} \cup \Sigma)^*$.

Condition (i) states the compatibility of the derived constituent with the island(s) found to its left. This is exemplified in Fig. C.1, where positions i_h and i_{h+1} indicate two selected islands, and where the portion $a_{i_h+1} \cdots a_{i_{h+1}}$ of the input string has been divided into the left substring $a_{i_h+1} \cdots a_{q_h}$ and the right substring $a_{q_h+1} \cdots a_{i_{h+1}}$ (we have also assumed $i_{h+1} \leq j$). Constituent A is not compatible with its left island(s) if its subconstituents within the substring $a_{i_h+1} \cdots a_{q_h}$ are not “reachable” from some partial analysis that includes (at least) island a_{i_h} . Note that if this is the case, clearly no derivation including constituent A can eventually derive the input string. Similarly, Condition (iii) (see Fig. C.2) states that a constituent A found in between two consecutive islands a_{i_h} and $a_{i_{h+1}}$ and dominating the boundary between them is not compatible with these surrounding islands if A is not “reachable” from

either some left or some right partial analysis that includes (at least) a_{i_h} or $a_{i_{h+1}}$ respectively.

The above compatibility relation is used in the next theorem in order to characterize the set of all i-items inserted in T by Algorithm 12.

Theorem 17. *For every $A \in I_G^{(I)}$ and for every i and j , $0 \leq i \leq j \leq n$, the following conditions hold:*

- (i) *if $A \in t_{i,j}$ then there exists a derivation ρ of the form $A \xRightarrow{*}_{G_I} a_{i+1} \cdots a_j$ such that $R_w^{(I)}(\rho)$;*
- (ii) *if there exists a derivation ρ of the form $A \xRightarrow{*}_{G_I} a_{i+1} \cdots a_j$ such that $R_w^{(I)}(\rho)$, then there exists $B \in I_G^{(I)}$ such that $(A, B) \in S_I$ and either $B \in t_{i-d,j}$ or $B \in t_{i,j+d}$ for some integer $d \geq 0$. $\square$*

Comparing Theorem 17 with the general condition reported in (1) (Section 2.1) we see that Algorithm 12 is more restrictive than a pure bottom-up recognizer: in fact relation $R_w^{(I)}$ is effective in filtering out partial analyses based on the context of the surrounding islands. It is also interesting to compare the above invariant with the valid prefix property mentioned in Section 2.3 for the Earley algorithm. As a result of the island-outward requirement and the top-down prediction, we obtain some sort of “infix” property, where the islands surrounding a given constituent play the role of the beginning of the string in the valid prefix property. Finally, observe that, as in the case of the head-driven recognizer, the subsumption blocking technique adopted by Algorithm 12 does not guarantee that a partial analysis deriving a substring of the input including some island is directly recognized. We are guaranteed, however, that a second partial analysis “subsuming” the first one will always be inserted in the recognition matrix: relation S_I has been used in Condition (ii) of the invariant to account for this fact.

Acknowledgments

We are indebted to Anna Corazza, Alberto Lavelli, René Leermakers, Fernando Pereira and Klaas Sikkels for their valuable comments on a previous version of this manuscript. Any error in this work should be blamed only on us.

Giorgio Satta was partially funded by the following grants: ARO grant DAAL 03-89-C-0031, DARPA grant N00014-90-J-1863, NSF grant IRI 90-16592 and Ben Franklin grant 91S.3078C-1.

References

- [1] A.V. Aho, J.E. Hopcroft and J.D. Ullman, *The Design and Analysis of Computer Algorithms* (Addison-Wesley, Reading, MA, 1974).

- [2] A.V. Aho and J.D. Ullman, *The Theory of Parsing, Translation and Compiling, Vol. 1* (Prentice-Hall, Englewood Cliffs, NJ, 1972).
- [3] K. Ajdukiewicz, Die syntaktische Konnexität, *Stud. Philos.* **1** (1935) 1–27; English translation by H. Weber in: S. McCall, ed., *Polish Logic 1920–1939* (Oxford University Press, Oxford, 1967) 207–231.
- [4] A. Bossi, N. Cocco and L. Colussi, A divide-and-conquer approach to general context-free parsing, *Inf. Process. Lett.* **16** (1983) 203–208.
- [5] N. Chomsky, *Aspects of the Theory of Syntax* (MIT Press, Cambridge, MA, 1965).
- [6] L. Devos and M. Gilloux, GPSG parsing, bidirectional charts, and connection graphs, in: *Proceedings 13th COLING*, Helsinki, Finland (1990) 151–155.
- [7] J.P.M. de Vreught and H.J. Honig, A tabular bottom-up recognizer, Tech. Report 89-78, Faculty of Technical Mathematics and Informatics, Delft University of Technology, Delft, Netherlands (1989).
- [8] J. Earley, An efficient context-free parsing algorithm, Ph.D. Thesis, Department of Computer Science, Carnegie-Mellon University, Pittsburgh, PA (1968).
- [9] J. Earley, An efficient context-free parsing algorithm, *Commun. ACM* **13** (2) (1970) 94–102.
- [10] G. Gazdar, E. Klein, G. Pullum and I. Sag, *Generalized Phrase Structure Grammar* (Harvard University Press, Cambridge, MA, 1985).
- [11] E.P. Giachin and C. Rullent, A parallel parser for spoken natural language, in: *Proceedings IJCAI-89*, Detroit, MI (1989) 1537–1542.
- [12] E. Gibson, Bidirectional active chart parsing, Tech. Report CMU-CMT-93-139, Center for Machine Translation, Carnegie Mellon University, Pittsburgh, PA (1993).
- [13] S.L. Graham and M.A. Harrison, Parsing of general context free languages, in: *Advances in Computers* **14** (Academic Press, New York, NY, 1976) 77–185.
- [14] S.L. Graham, M.A. Harrison and W.L. Ruzzo, An improved context-free recognizer, *ACM Trans. Programm. Lang. Syst.* **2** (3) (1980) 415–462.
- [15] M.A. Harrison, *Introduction to Formal Language Theory* (Addison-Wesley, Reading, MA, 1978).
- [16] R. Jackendoff, *X-bar Syntax: A Study of Phrase Structure* (MIT Press, Cambridge, MA, 1977).
- [17] A.K. Joshi, How much context-sensitivity is necessary for characterizing structural descriptions—Tree Adjoining Grammars, in: D. Dowty, L. Karttunen and A. Zwicky, eds., *Natural Language Processing—Theoretical, Computational and Psychological Perspectives* (Cambridge University Press, New York, 1985); originally presented in a Workshop on Natural Language Parsing at Ohio State University, Columbus, OH (1983).
- [18] R. Kaplan, A general syntactic processor, in: E. Rustin, ed., *Natural Language Processing* (Prentice-Hall, Englewood Cliffs, NJ, 1973).
- [19] M. Kay, Algorithm schemata and data structures in syntactic processing, Tech. Report CSL-80, Xerox Palo Alto Research Center, Palo Alto, CA (1980); also in: B.J. Grosz, K. Sparck Jones and B.L. Webber, eds., *Natural Language Processing* (Morgan Kaufmann, Los Altos, CA, 1986) 35–70.
- [20] M. Kay, Head-driven parsing, in: *Proceedings Workshop on Parsing Technologies*, Pittsburgh, PA (1989) 52–62.
- [21] A. Lavelli and G. Satta, Bidirectional parsing of lexicalized tree adjoining grammars, in: *Proceedings Fifth Meeting of the European Association for Computational Linguistics*, Berlin, Germany (1991).
- [22] R. Leermakers, How to cover a grammar, in: *Proceedings 27th Annual Meeting of the Association for Computational Linguistics*, Vancouver, BC (1989) 135–142.
- [23] A. Nijholt, *Context-free Grammars: Covers, Normal Forms, and Parsing* (Springer, Berlin, Germany, 1980).
- [24] A. Nijholt, The CYK-approach to serial and parallel parsing, Memoranda Informatica 90-13, Department of Computer Science, University of Twente, Enschede, Netherlands (1990).
- [25] F.C.N. Pereira and D.H.D. Warren, Parsing as deduction, in: *Proceedings 21th Annual Meeting of the Association for Computational Linguistics*, Cambridge, MA (1983) 137–144.

- [26] C. Pollard and I. Sag, *Information-Based Syntax and Semantics, Vol. 1: Fundamentals*, CSLI Lecture Notes Series 13 (Center for the Study of Language and Information, Stanford University, Stanford, CA, 1987).
- [27] D. Proudian and C. Pollard, Parsing head-driven phrase structure grammar, in: *Proceedings 23rd Annual Meeting of the Association for Computational Linguistics*, Chicago, IL (1985) 167–171.
- [28] D.J. Rosenkrantz and P.M. Lewis, Deterministic left corner parsing, in: *IEEE Conference Record 11th Annual Symposium on Switching and Automata Theory* (1970) 139–152.
- [29] H. Saito, Bi-directional LR parsing from an anchor word for speech recognition, in: *Proceedings 13th COLING*, Helsinki, Finland (1990) 237–242.
- [30] G. Satta and O. Stock, Formal properties and implementation of bidirectional charts, in: *Proceedings IJCAI-89*, Detroit, MI (1989) 1480–1485.
- [31] G. Satta and O. Stock, Head-driven bidirectional parsing: a tabular method, in: *Proceedings Workshop on Parsing Technologies*, Pittsburgh, PA (1989) 43–51.
- [32] G. Satta and O. Stock, A tabular method for island-driven context-free grammar parsing, in: *Proceedings AAAI-91*, Anaheim, CA (1991) 143–148.
- [33] Y. Schabes, The valid prefix property and left to right parsing of tree-adjoining grammar, in: *Proceedings Second International Workshop on Parsing Technologies*, Cancun, Mexico (1991).
- [34] B.A. Sheil, Observations on context-free parsing, in: *Statistical Methods in Linguistics*. Stockholm, Sweden (1976); also: Tech. Report TR 12-76, Center for Research in Computing Technology, Aiken Computation Laboratory, Harvard University, Cambridge, MA (1976).
- [35] S.M. Shieber, Using restriction to extend parsing algorithms for complex-feature-based formalisms, in: *Proceedings 23rd Annual Meeting of the Association for Computational Linguistics*, Chicago, IL (1985) 145–152.
- [36] S.M. Shieber, *An Introduction to Unification-Based Approaches to Grammar*, CSLI, Stanford, CA (1986).
- [37] S.M. Shieber, G. van Noord, R.C. Moore and F.C.N. Pereira, A semantic-head-driven generation algorithm for unification-based formalisms, in: *Proceedings 27th Annual Meeting of the Association for Computational Linguistics*, Vancouver, BC (1989) 7–17.
- [38] K. Sikkil and R. op den Akker, Head-corner chart parsing, in: J. Dietz, ed., *Proceedings Computing Science in the Netherlands, Utrecht* (SION, Centrum voor Wiskunde en Informatica, Amsterdam, 1992) 279–290.
- [39] S. Sippu and E. Soisalon-Soininen, *Parsing Theory: Languages and Parsing, Vol. 1*, (Springer, Berlin, 1988).
- [40] S. Steel and A.D. Roeck, Bidirectional parsing, in: Hallam and C. Mellish, eds., *Advances in AI, Proceedings 1987 AISB Conference* (Wiley, London, 1987) 223–235.
- [41] O. Stock, Parsing with flexibility, dynamic strategies, and idioms in mind, *Comput. Linguist.* 15 (1) (1989) 1–18.
- [42] O. Stock, R. Falcone and P. Insinnamo, Bidirectional charts: a potential technique for parsing spoken natural language sentences, *Comput. Speech Lang.* 3 (3) (1989) 219–237.
- [43] L.G. Valiant, General context-free recognition in less than cubic time, *J. Comput. Syst. Sci.* 10 (1975) 219–237.
- [44] G. van Noord, An overview of head-driven bottom-up generation, in: R. Dale, C. Mellish and M. Zock, eds., *Current Research in Natural Language Generation* (Academic Press, London, 1990) 141–165.
- [45] D.E. Walker, ed., *Understanding Spoken Language* (North-Holland, New York, 1978).
- [46] M. Wren, A comparison of rule-invocation strategies in parsing, in: *Proceedings Third Meeting of the European Association for Computational Linguistics*, Copenhagen, Denmark (1987) 226–233.
- [47] W. Woods, M. Bates, G. Brown, B. Bruce, C. Cook, J. Klovstad, J. Makhoul, B. Nash-Webber, R. Schwartz, J. Wolf and V. Zue, Speech understanding systems—final technical report, Tech. Report 3438 (Bolt, Beranek and Newman, Cambridge, MA, 1976).

- [48] W.A. Woods, Optimal search strategies for speech understanding control, *Artif. Intell.* **18** (1982) 295–326.
- [49] W.A. Woods, Language processing for speech understanding, in: F. Fallside and W.A. Woods, eds., *Computer Speech Processing* (Prentice Hall, Englewood Cliffs, NJ, 1985) Chapter 12, 305–334.
- [50] D.H. Younger, Recognition and parsing of context-free languages in time n^3 , *Inf. Control* **10** (1967) 189–208.